Convex Hulls

Sujoy Bhore

Indian Institute of Technology Bombay

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%

Can we mix ... ?

q_1	15%	20%
q_2	25%	28%

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%

Can we mix ... ?

q_1	15%	20%
q_2	25%	28%

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%
S_3	40%	25%

Can we mix ... ?

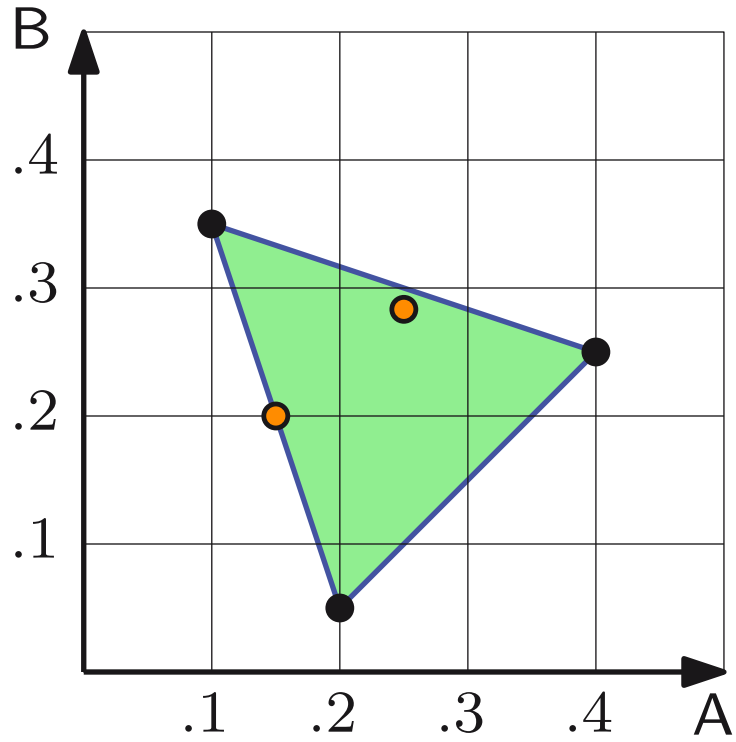
q_1	15%	20%
q_2	25%	28%

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%
S_3	40%	25%

Geometric representation



Can we mix ... ?

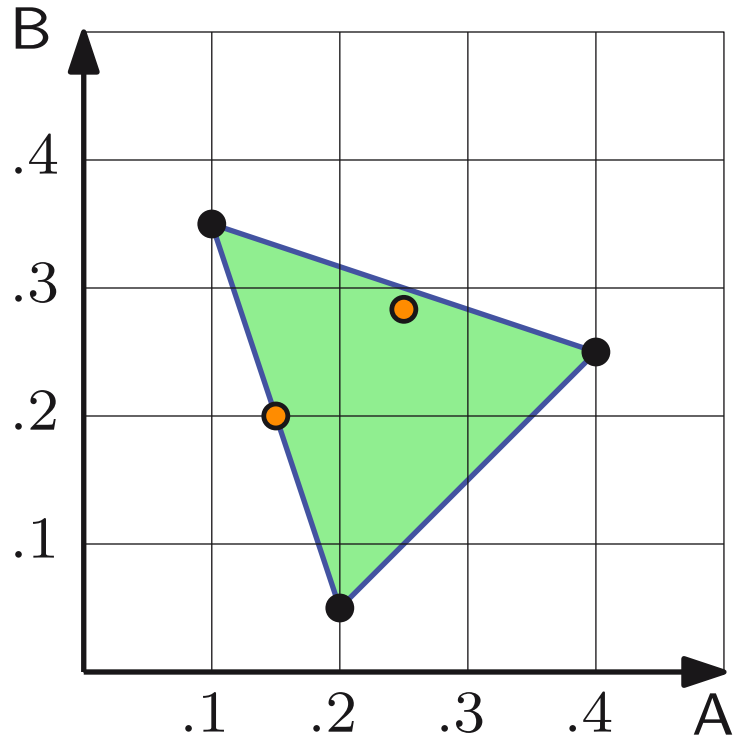
q_1	15%	20%
q_2	25%	28%

Mixing Ratios

We are given -

Mixture	fraction A	fraction B
S_1	10%	35%
S_2	20%	5%
S_3	40%	25%

Geometric representation



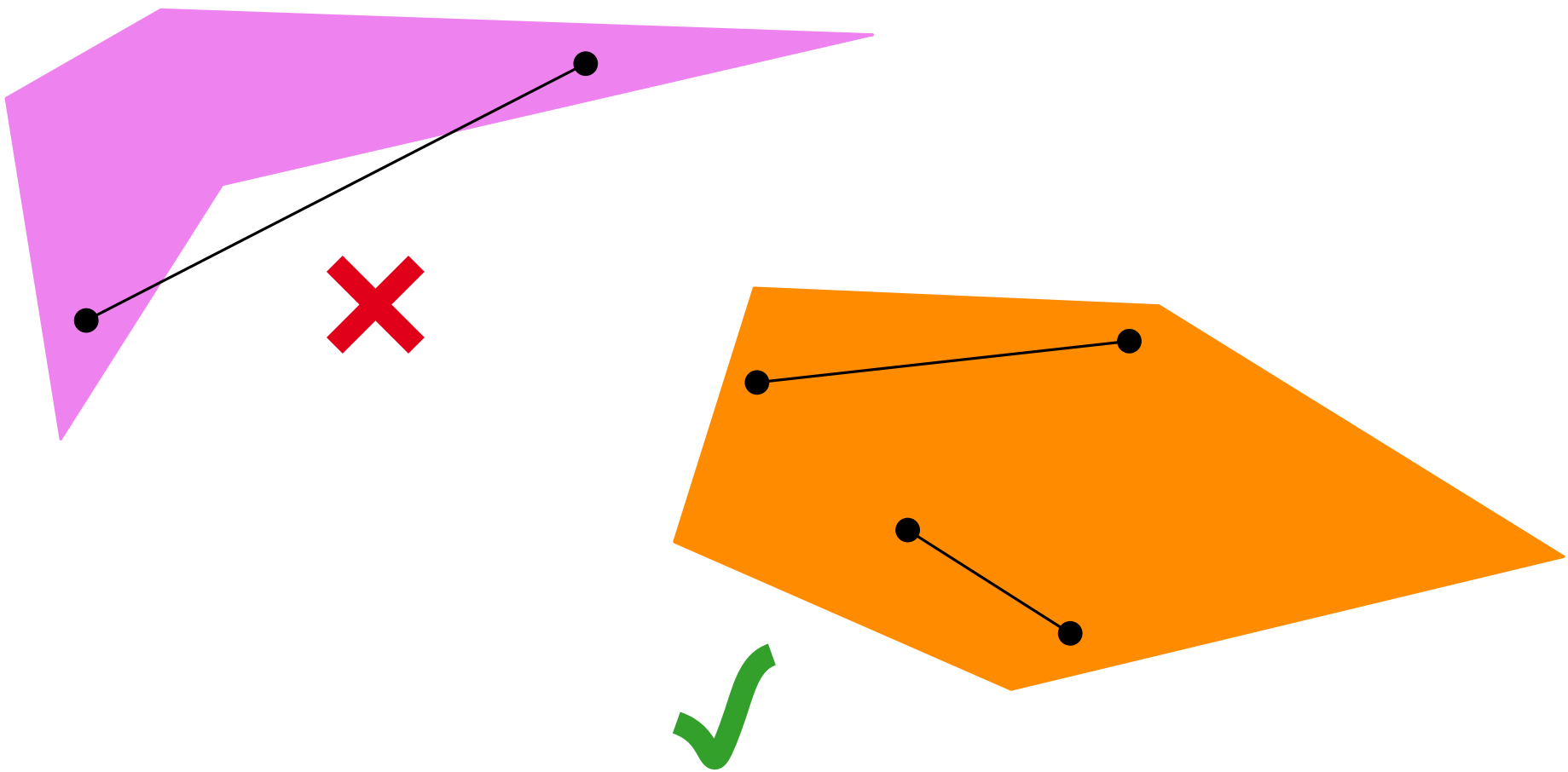
Can we mix ... ?

q_1	15%	20%
q_2	25%	28%

Observation. Given a set $S \subseteq \mathbb{R}^2$ mixtures, it is possible to make another mixture $q \in \mathbb{R}^2$ using $S \iff q \in$ ConvexHull $CH(S)$.
$q = \sum_i \lambda_i s_i$ with $\sum_i \lambda_i = 1$.

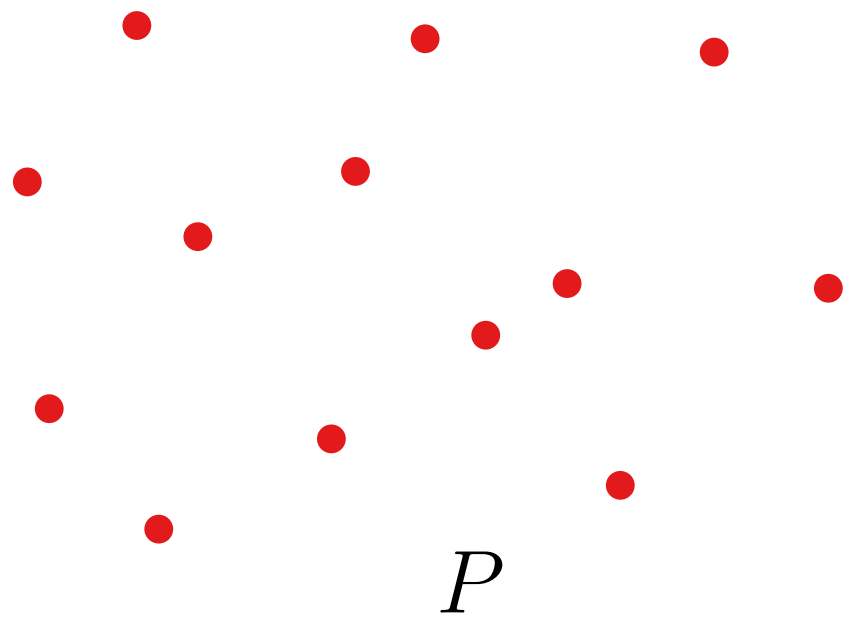
Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.



Definition of Convex Hull

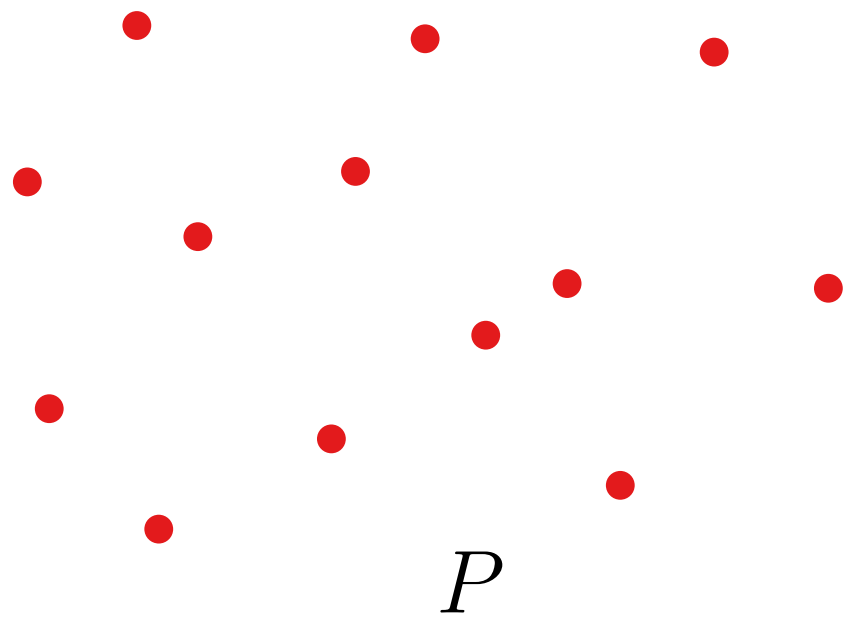
Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.



Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.

How about some physics -

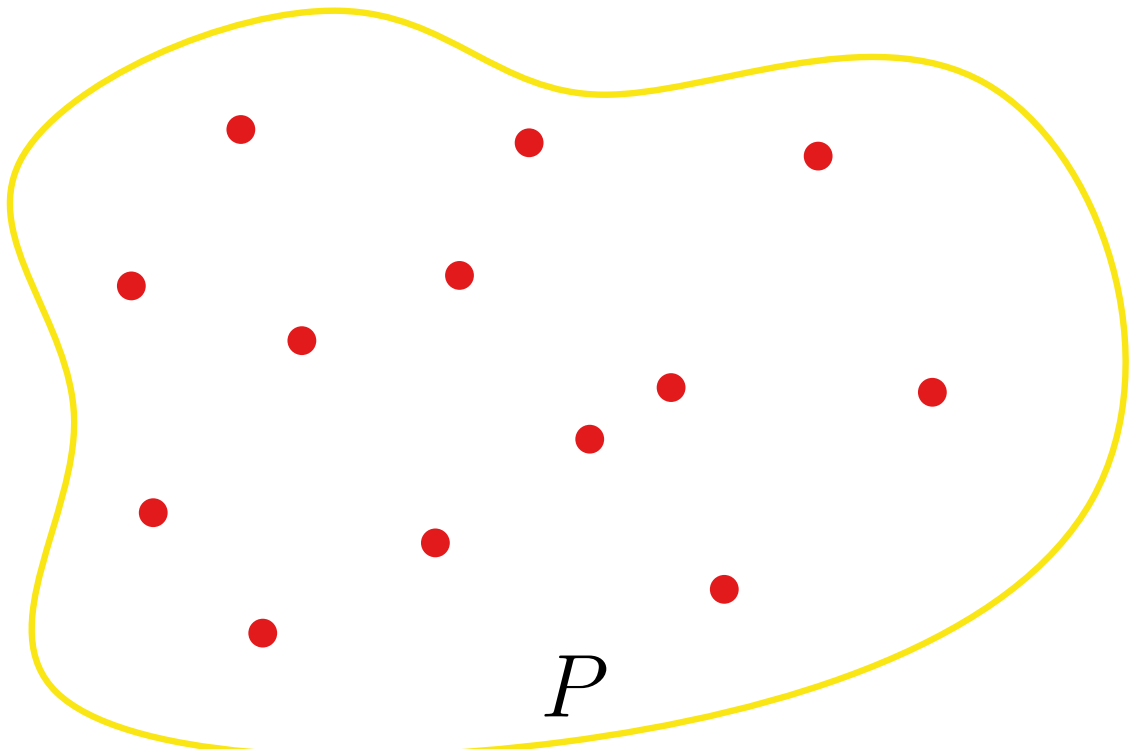


Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.

How about some physics -

- put a large rubber band around all points

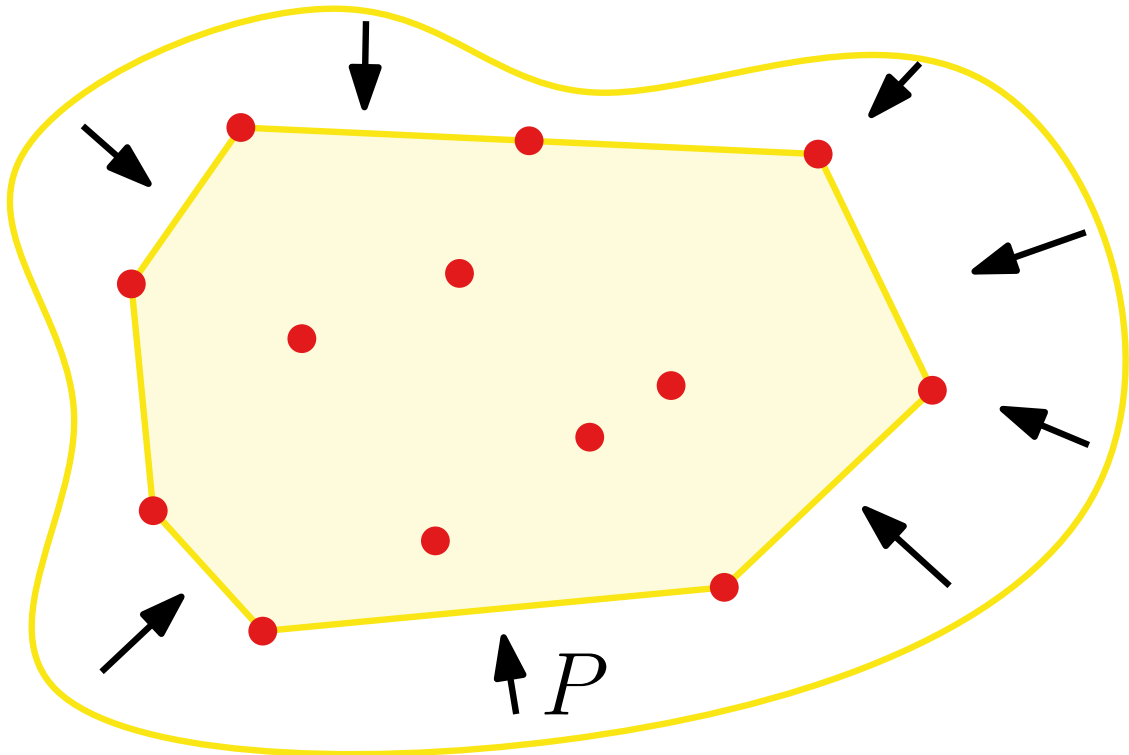


Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.

How about some physics -

- put a large rubber band around all points
- and let it go!

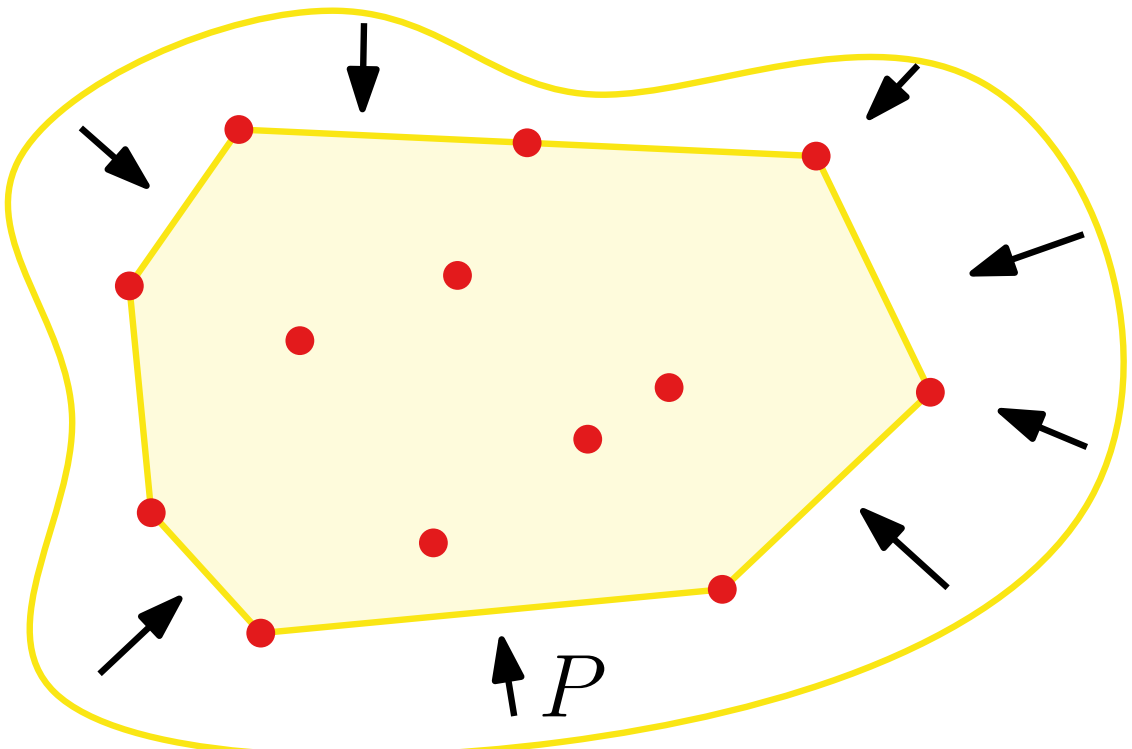


Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.

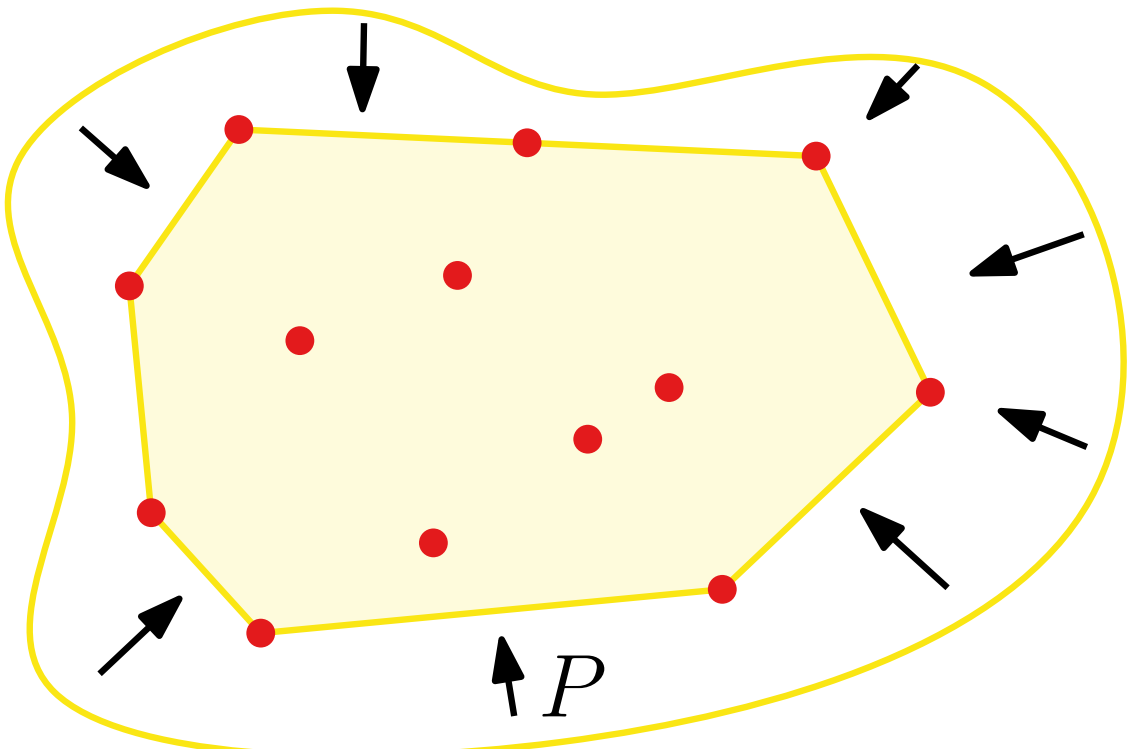
How about some physics -

- put a large rubber band around all points
- and let it go!
- unfortunately, does not help algorithmically



Definition of Convex Hull

Definition. A region $S \subseteq \mathbb{R}^2$ is called **convex**, if for any two points $p, q \in S$ the line segment $\overline{pq} \in S$. The **convex hull** $CH(S)$ of S is the smallest convex region containing S.



How about some physics -

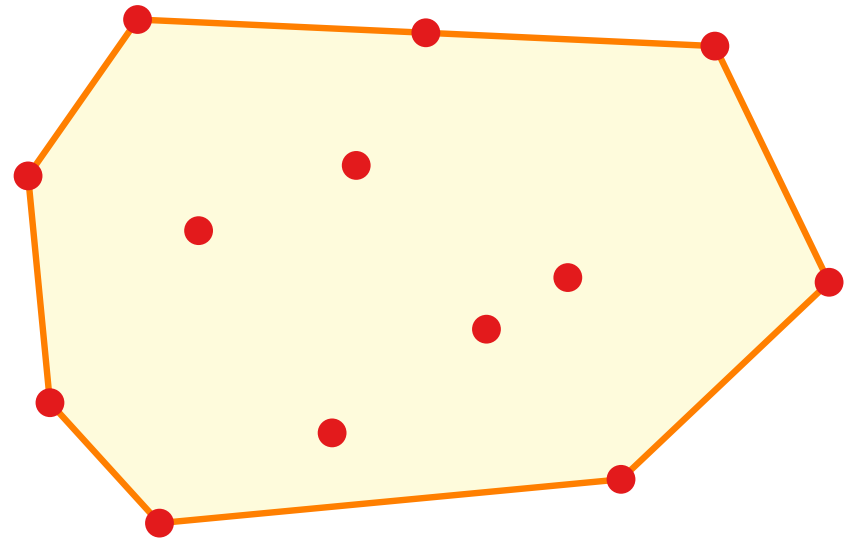
- put a large rubber band around all points
- and let it go!
- unfortunately, does not help algorithmically

Now, some Mathematics -

- define $CH(S) = \bigcap_{C \supseteq S:\ C \text{ convex}} C$
- does not help either.

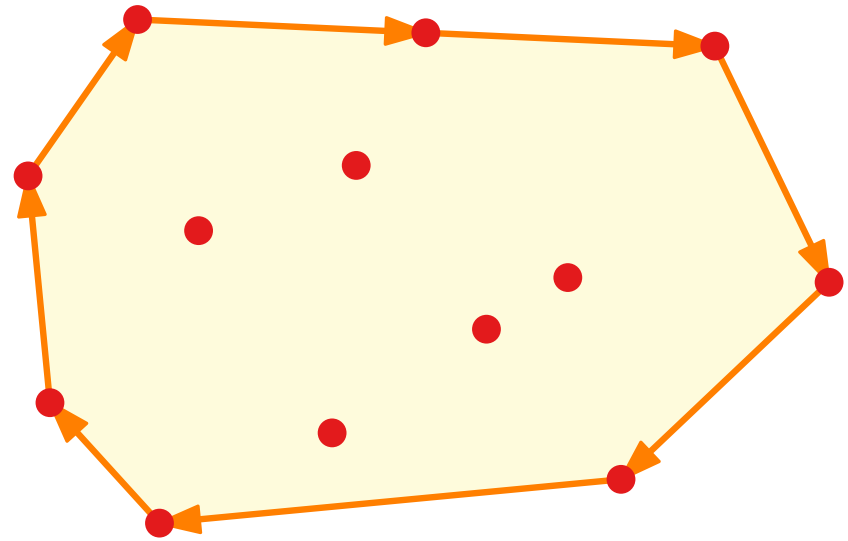
Algorithmic Approach

Lemma. For a set of points $P \subseteq \mathbb{R}^2$, $CH(P)$ is a convex polygon that contains P and whose vertices are a subset of P.



Algorithmic Approach

Lemma. For a set of points $P \subseteq \mathbb{R}^2$, $CH(P)$ is a convex polygon that contains P and whose vertices are a subset of P.

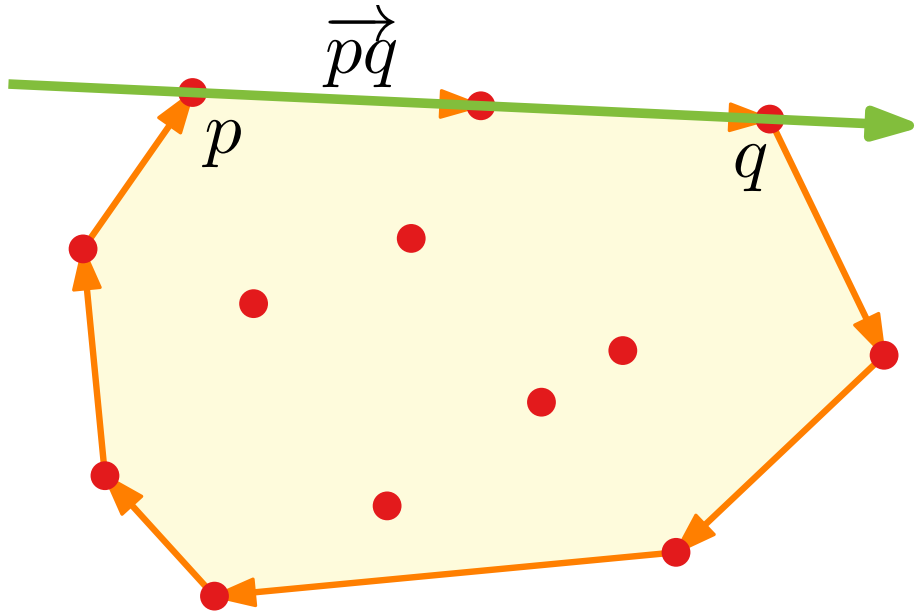


Input: A set of points $P = \{p_1, \ldots, p_n\}$

Output: List of vertices of $CH(P)$ in clockwise order

Algorithmic Approach

Lemma. For a set of points $P \subseteq \mathbb{R}^2$, $CH(P)$ is a convex polygon that contains P and whose vertices are a subset of P.



Input: A set of points $P = \{p_1, \ldots, p_n\}$

Output: List of vertices of $CH(P)$ in clockwise order

Observation. (p, q) is an edge of $CH(P) \Leftrightarrow$ each point $r \in P \setminus \{p, q\}$

- strictly right of the oriented line $\overrightarrow{pq}$ or
- on the line segment $\overline{pq}$

Algorithm - 1

ConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E

return L

Algorithm - 1

ConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

Check all possible edges (p, q)

    valid $\leftarrow$ *true*

    foreach $r \in P$ **do**

        if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

            valid $\leftarrow$ *false*

    if *valid* **then**

        $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E

return L

Algorithm - 1

ConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

Check all possible edges (p, q)

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

Test in $O(1)$ time with

$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\rightarrow$ think (exercise-1)

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** $\mathcal{O}(1)$

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do**

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

$\Theta(1)$

$\Theta(n)$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** —— $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

$\Theta(1)$ $\Theta(n)$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** —— $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

$\Theta(1)$ $\Theta(n)$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** —— $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

      valid $\leftarrow$ *false*

  if *valid* **then**

    $E \leftarrow E \cup \{(p, q)\}$

$\Theta(1)$ $\Theta(n)$

construct sorted vertex list L of $CH(P)$ from E

return L

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** — $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** — $\Theta(1)$

      valid $\leftarrow$ *false*

  if *valid* **then** — $\Theta(n)$

    $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E — $O(n^2)$

return L

$\Theta(n^3)$

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** — $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** — $\Theta(1)$

      valid $\leftarrow$ *false*

  if *valid* **then** — $\Theta(n)$

    $E \leftarrow E \cup \{(p, q)\}$

construct sorted vertex list L of $CH(P)$ from E — $O(n^2)$

return L

$\Theta(n^3)$

Lemma. The convex hull of n points in $\mathbb{R}^2$ can be computed in $O(n^3)$ time.

Running Time Analysis

FirstConvexHull(P)

$E \leftarrow \emptyset$

foreach $(p, q) \in P \times P$ with $p \neq q$ **do** — $(n^2 - n)\cdot$

  valid $\leftarrow$ *true*

  foreach $r \in P$ **do**

    if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** — $\Theta(1)$

      valid $\leftarrow$ *false*

  if *valid* **then** — $\Theta(n)$

    $E \leftarrow E \cup \{(p, q)\}$

Can we do better?

construct sorted vertex list L of $CH(P)$ from E — $O(n^2)$

return L

$\Theta(n^3)$

Lemma. The convex hull of n points in $\mathbb{R}^2$ can be computed in $O(n^3)$ time.

How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

How about an Incremetal Approach ... ?

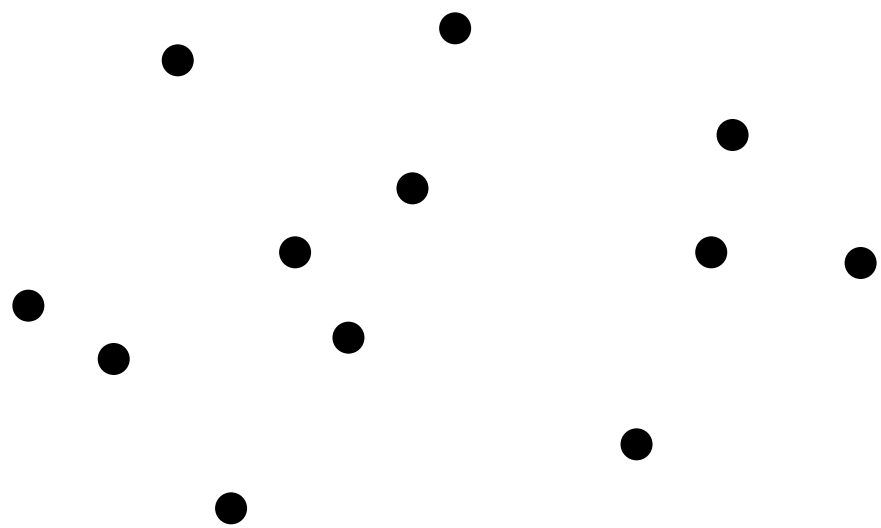
Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

Question: Which ordering of the points is useful?
Answer: From left to right!...?

How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

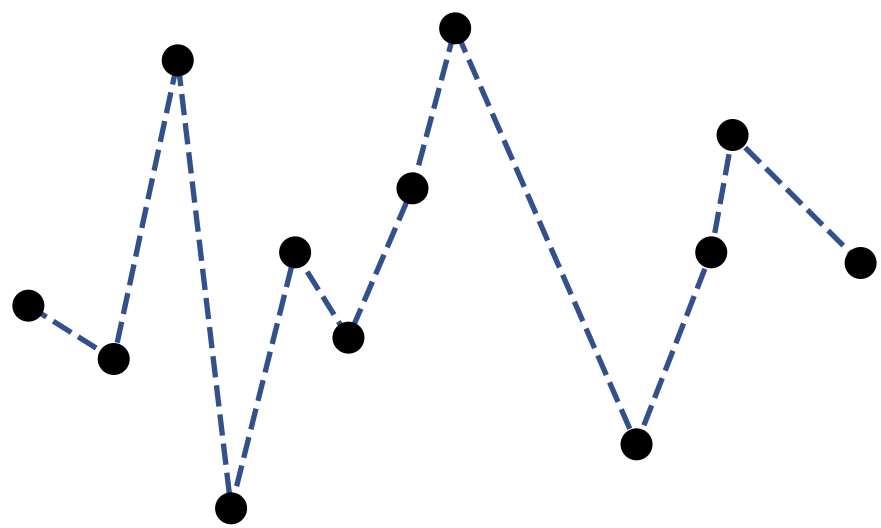
Question: Which ordering of the points is useful?
Answer: From left to right!...?



How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

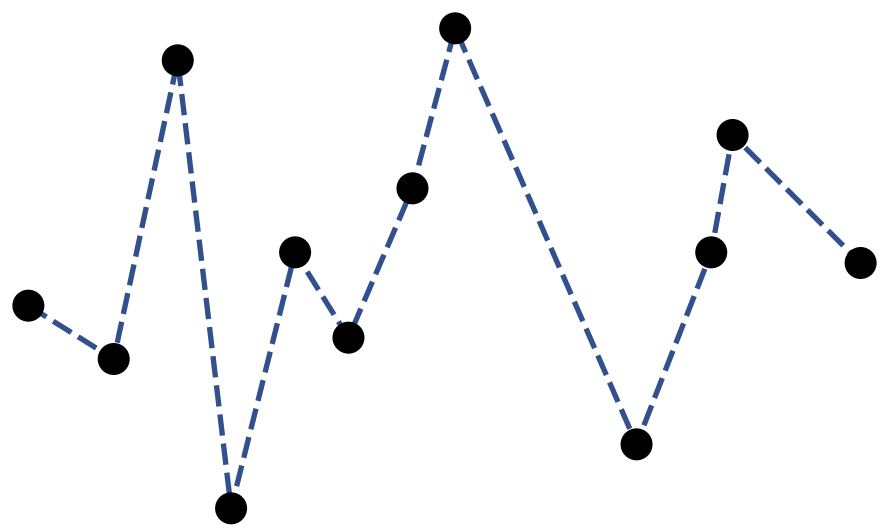
Question: Which ordering of the points is useful?
Answer: From left to right!...?



How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

Question: Which ordering of the points is useful?
Answer: From left to right!...?

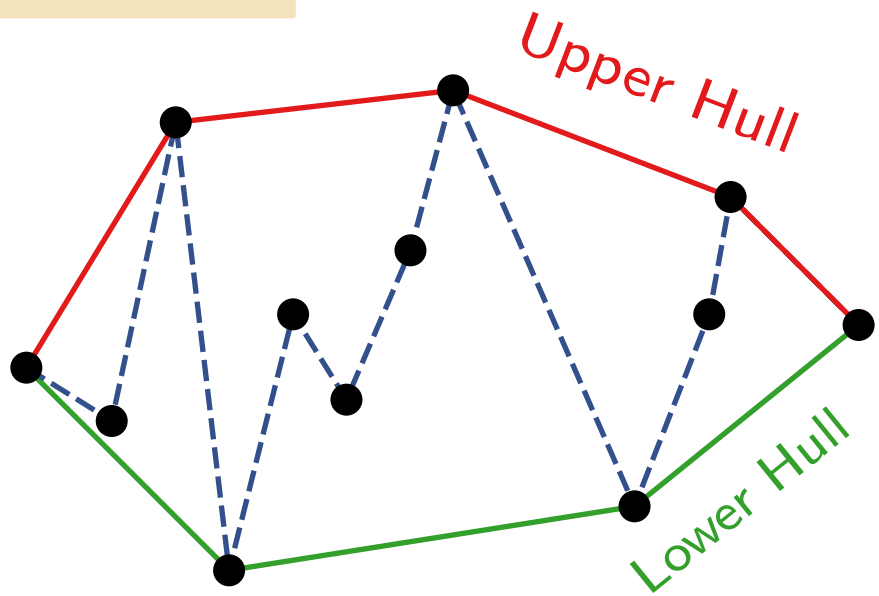


How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

Question: Which ordering of the points is useful?
Answer: From left to right!...?

Consider the Upper & Lower hulls separately.

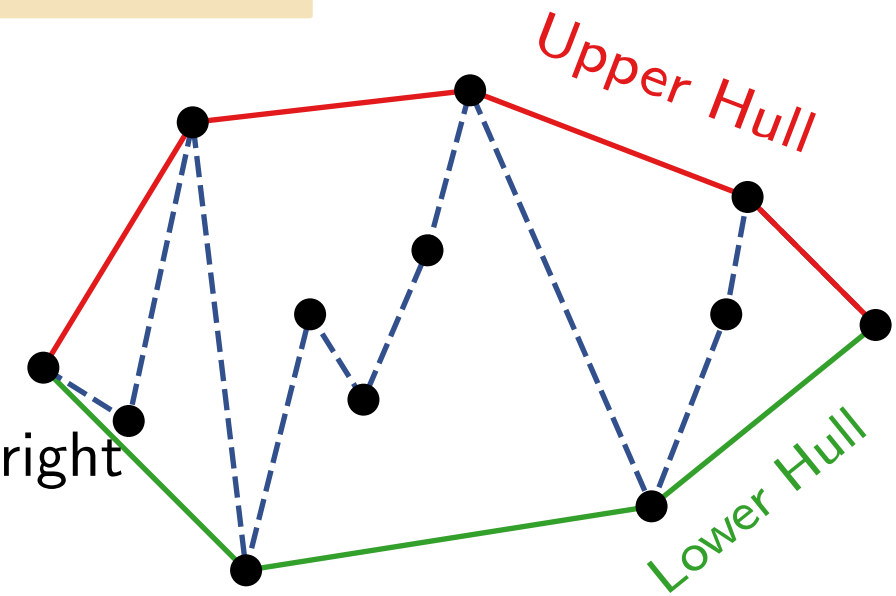


How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

Question: Which ordering of the points is useful?
Answer: From left to right!...?

Consider the Upper & Lower hulls separately.



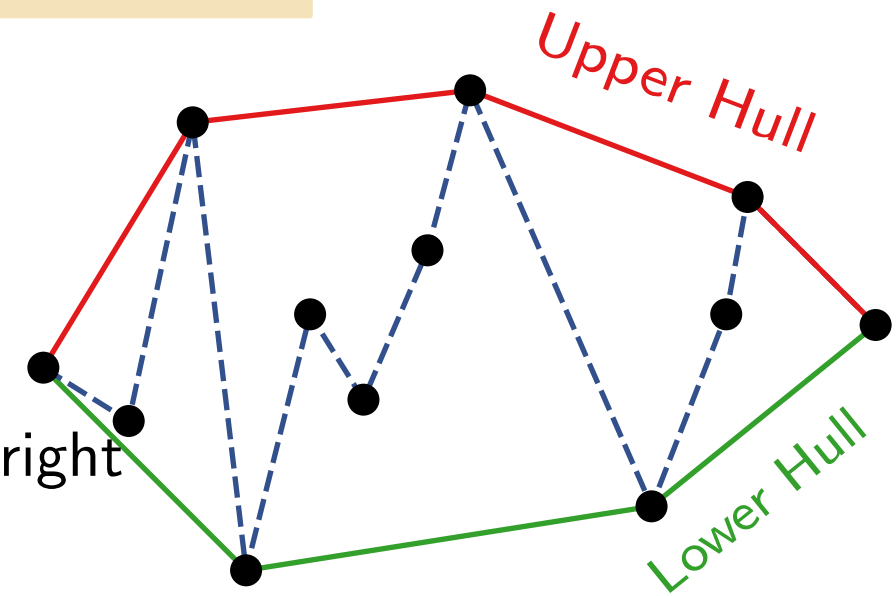
UpperConvexHull(P)
  $\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right
  $L \leftarrow \langle p_1, p_2 \rangle$
  for $i \leftarrow 3$ **to** n **do**
    L.append(p_i)
    while $|L| > 2$ **and** last 3 points in L do not form right turn **do**
      remove the second-to-last point in L
  return L

How about an Incremetal Approach ... ?

Idea: For $i = 1, \ldots, n$, compute $CH(P_i)$, where $P_i = \{p_1, \ldots, p_i\}$.

Question: Which ordering of the points is useful?
Answer: From left to right!...?

Consider the Upper & Lower hulls separately.



UpperConvexHull(P)
  $\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right
  $L \leftarrow \langle p_1, p_2 \rangle$
  for $i \leftarrow 3$ **to** n **do**
    L.append(p_i)
    while $|L| > 2$ **and** last 3 points in L do not form right turn **do**
      remove the second-to-last point in L
  return L

lower hull handled similarly!

Analysis

Running Time -

UpperConvexHull(P)

$\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right

$L \leftarrow \langle p_1, p_2 \rangle$

for $i \leftarrow 3$ **to** n **do**

  L.append(p_i)

  while $|L| > 2$ **and** last 3 points in L do not form right turn **do**

    remove the second-to-last point in L

return L

Analysis

Running Time -

UpperConvexHull(P)

- $\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right — $O(n \log n)$
- $L \leftarrow \langle p_1, p_2 \rangle$
- **for** $i \leftarrow 3$ **to** n **do** — $n - 2$
  - L.append(p_i)
  - **while** $|L| > 2$ **and** last 3 points in L do not form right turn **do** — $O(n)$
    - remove the second-to-last point in L
- **return** L

Analysis

Running Time -

UpperConvexHull(P)

$\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right $\quad O(n \log n)$

$L \leftarrow \langle p_1, p_2 \rangle$

for $i \leftarrow 3$ **to** n **do** $\quad n-2$

  L.append(p_i)

  while $|L| > 2$ **and** last 3 points in L do not form right turn **do** $\quad O(n)$

    remove the second-to-last point in L

return L

Amortized Analysis

- Each point is inserted into L exactly once
- A point in L is removed at most once from L
- $\Rightarrow$ Running time of the **for** loop including the **while** loop is $O(n)$

Analysis

Running Time -

UpperConvexHull(P)

$\langle p_1, p_2, \ldots, p_n \rangle \leftarrow$ sort P from left to right $\quad O(n \log n)$

$L \leftarrow \langle p_1, p_2 \rangle$

for $i \leftarrow 3$ **to** n **do** $\quad n-2$

  L.append(p_i)

  while $|L| > 2$ **and** last 3 points in L do not form right turn **do** $\quad O(n)$

    remove the second-to-last point in L

return L

Amortized Analysis

- Each point is inserted into L exactly once
- A point in L is removed at most once from L
- $\Rightarrow$ Running time of the **for** loop including the **while** loop is $O(n)$

Theorem. The convex hull of n points in the plane can be computed in $O(n \log n)$ time. $\rightarrow$ *Graham's Scan*.

Let's do a Gift Wrapping approach ...

Idea: Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$
while true **do**
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$
  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$
return $(p_1, \ldots, p_{j+1})$

Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

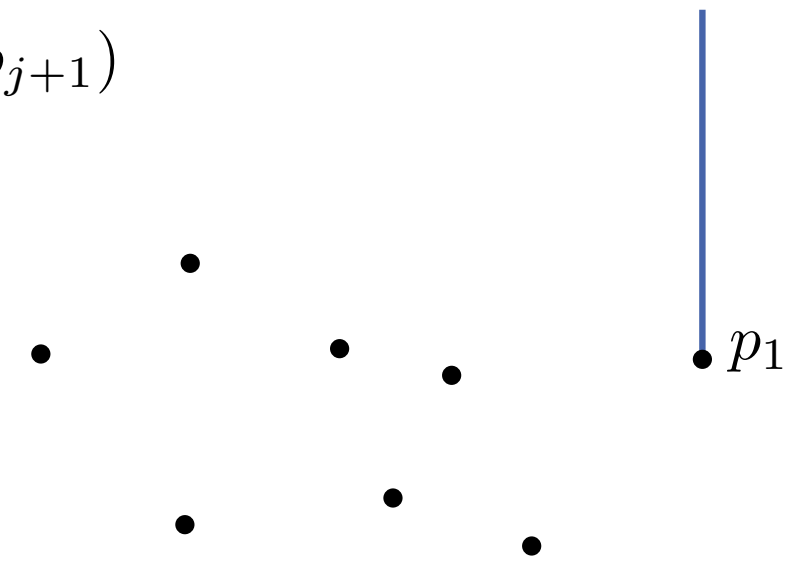
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

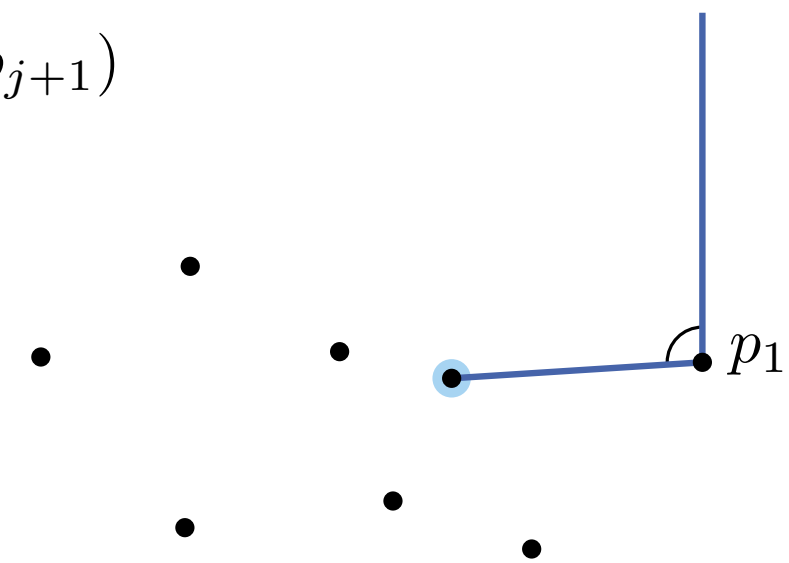


Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$
while true **do**
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$
  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$
return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

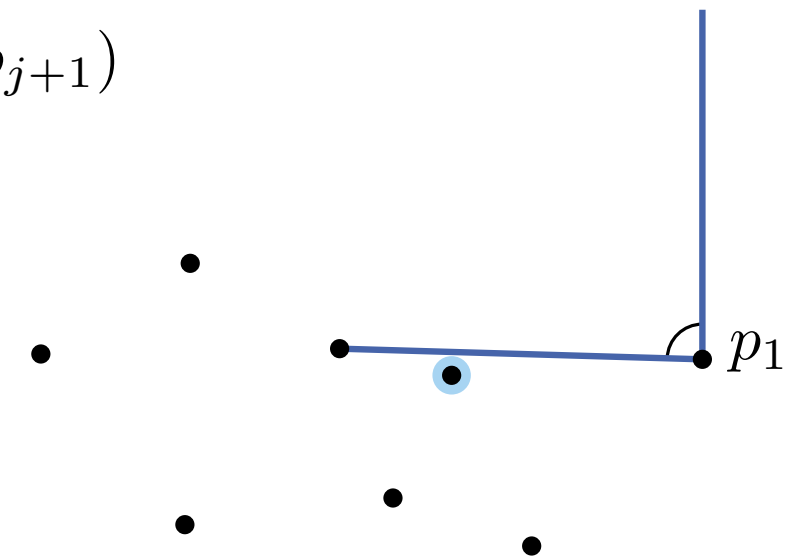
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

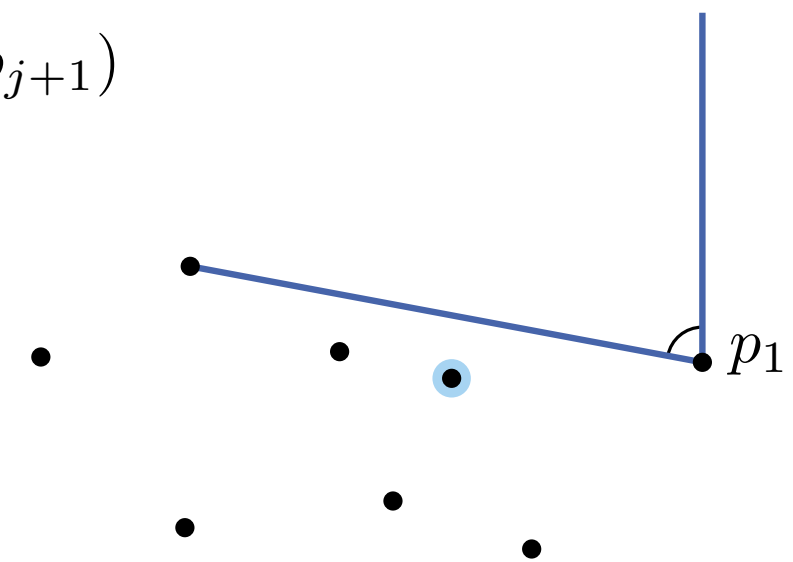
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

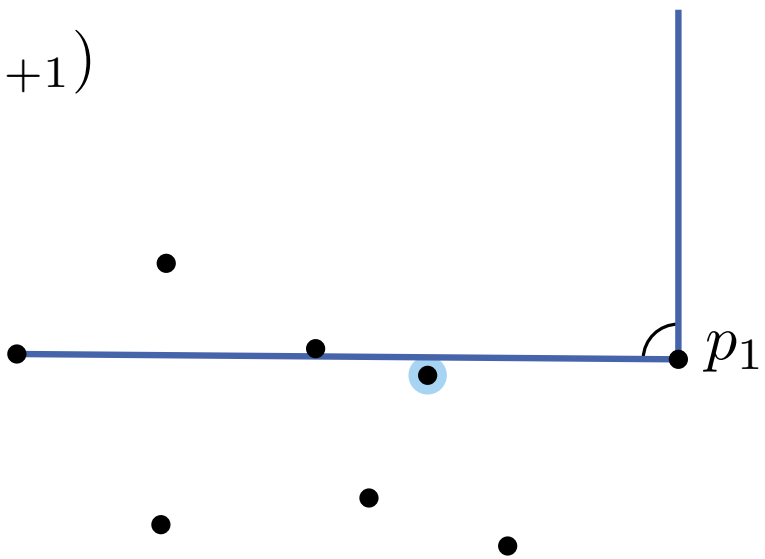


Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$
while true **do**
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$
  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$
return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

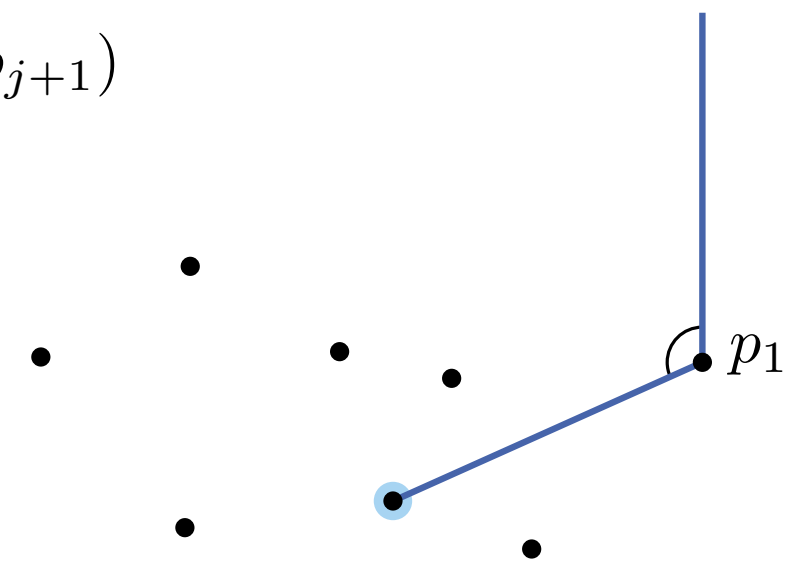
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

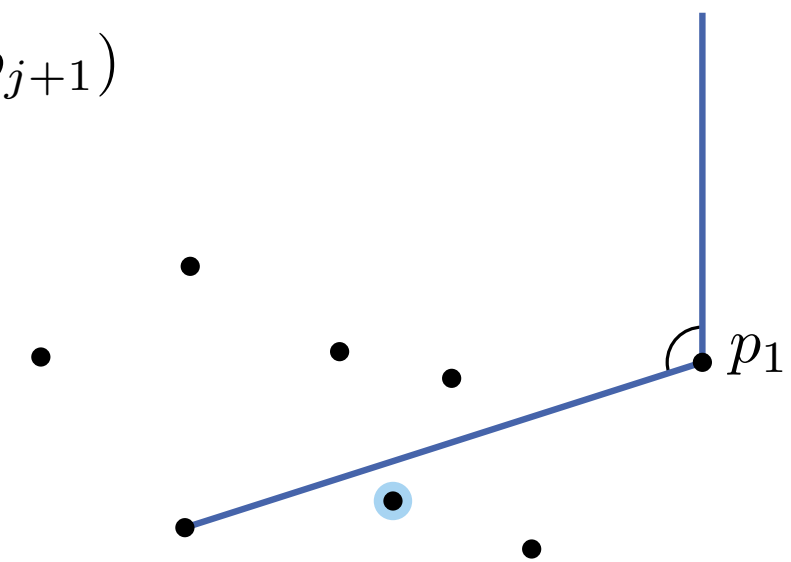
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

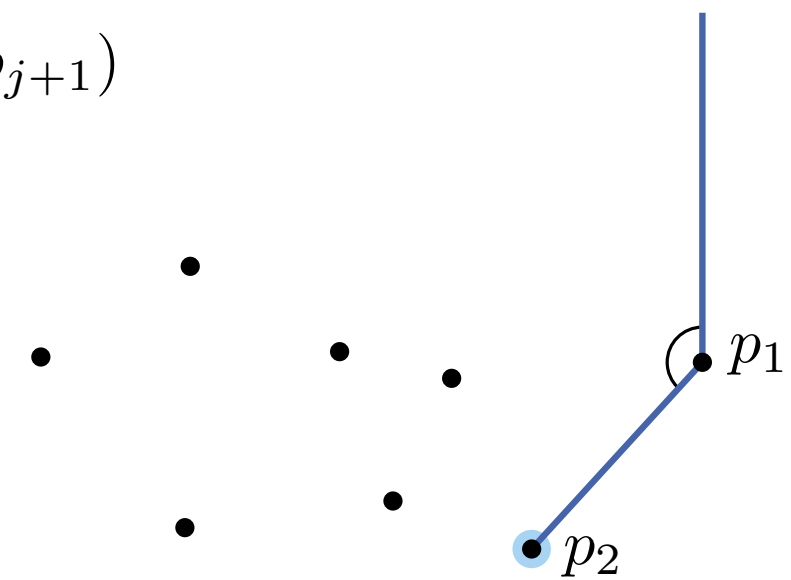
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

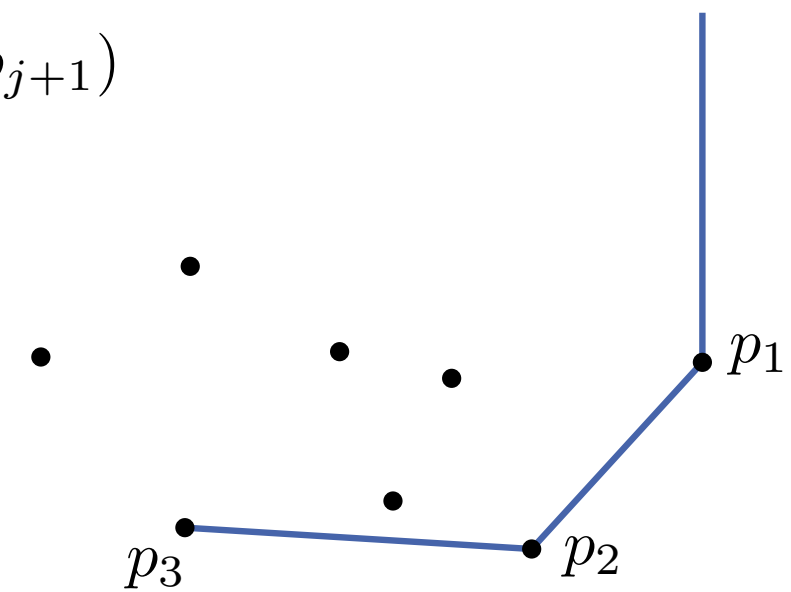


Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$
while true **do**
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$
  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$
return $(p_1, \ldots, p_{j+1})$

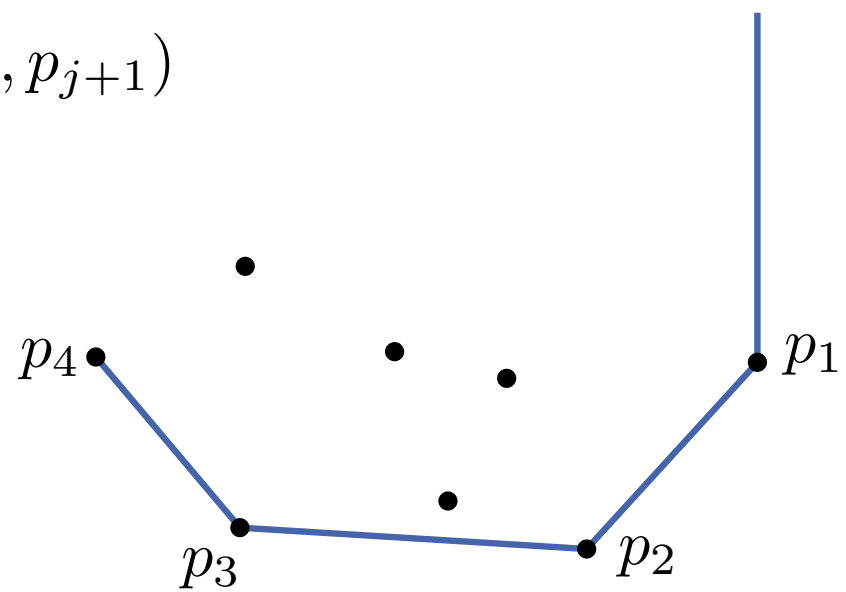


Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$
while true **do**
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$
  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$
return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

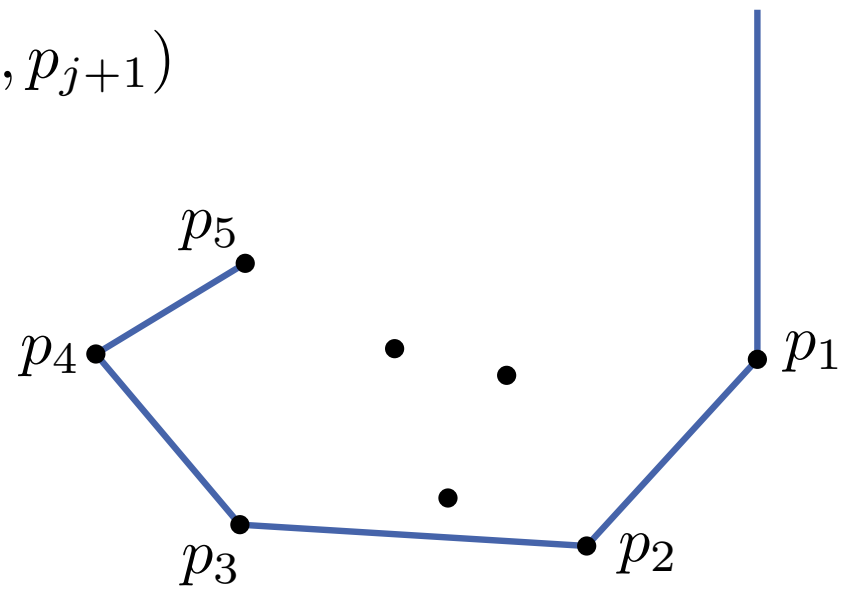
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

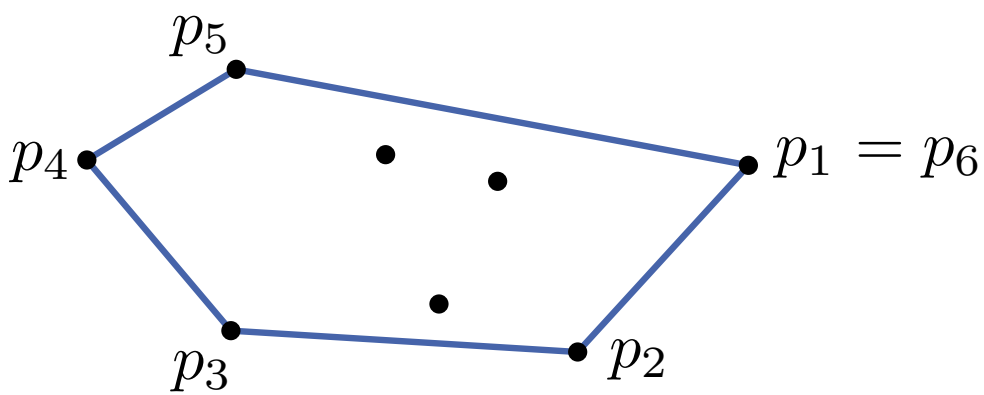
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

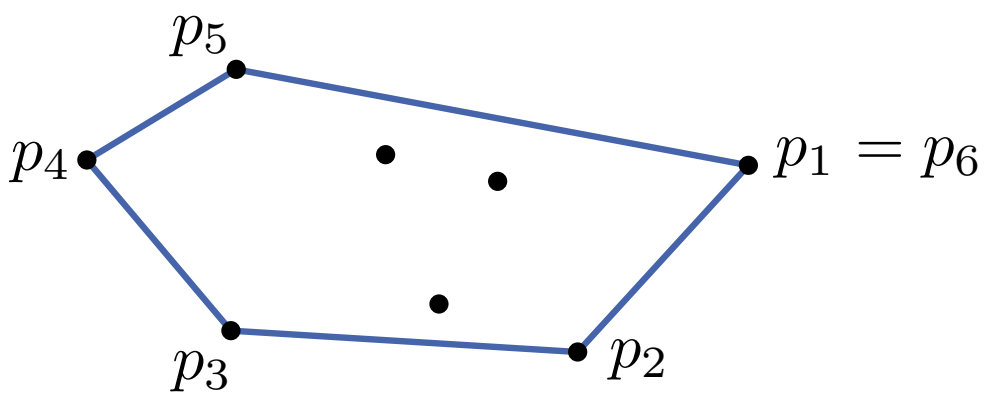
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

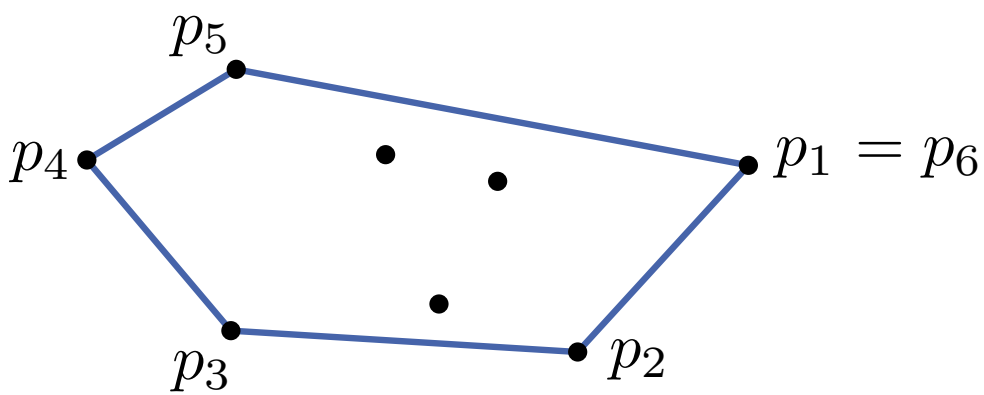
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do**

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$ $O(n)$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

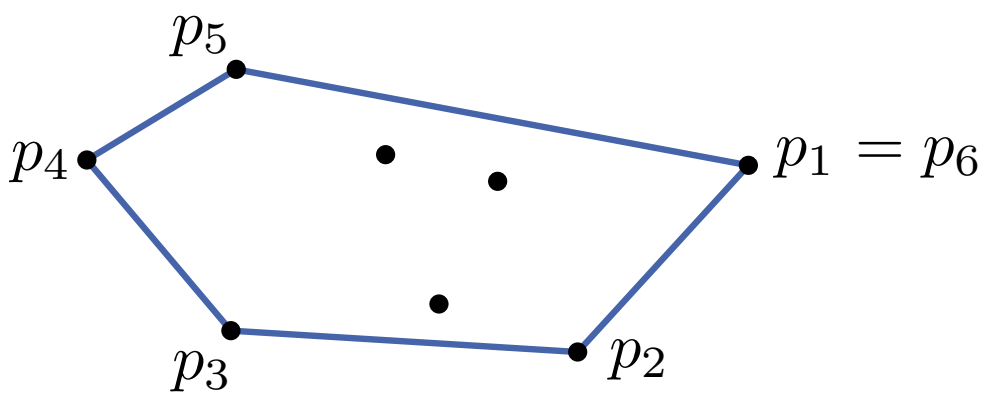
$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$

while true **do** — $O(h)$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$ — $O(n)$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$ — $O(n)$

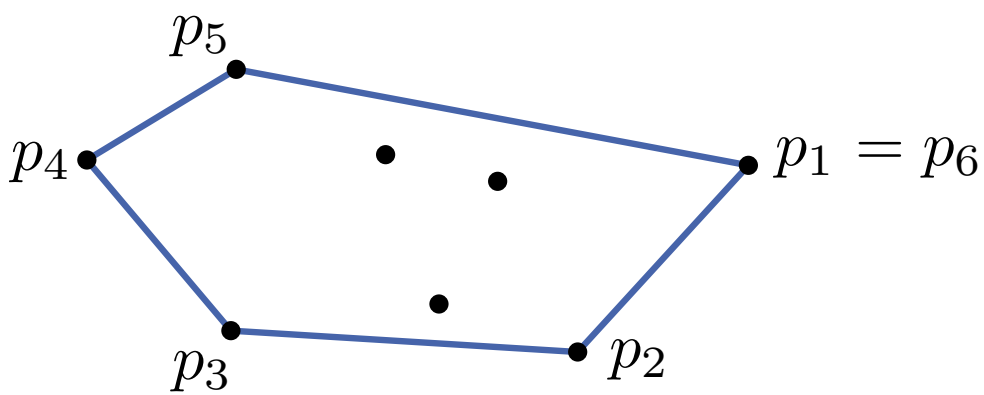
while true **do** — $O(h)$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$ — $O(n)$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

$O(n \cdot h)$



Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$ — $O(n)$

while true **do** — $O(h)$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$ — $O(n)$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

— $O(n \cdot h)$

return $(p_1, \ldots, p_{j+1})$

Theorem. The convex hull $CH(P)$ of n points P in $\mathbb{R}^2$ can be computed in $O(n \cdot h)$ time using *Gift Wrapping* (also called *Jarvis' March*), where $h = |CH(P)|$.

Alternative Approach: Gift Wrapping

Idea. Begin with a point p_1 of $CH(P)$, then find the next edge of $CH(P)$ in clockwise order.

GiftWrapping(P)

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P; $p_0 \leftarrow (x_1, \infty)$; $j \leftarrow 1$ — $O(n)$

while true **do** — $O(h)$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1}, p_j, q \mid q \in P \setminus \{p_{j-1}, p_j\}\}$ — $O(n)$

  if $p_{j+1} = p_1$ **then** break **else** $j \leftarrow j + 1$

return $(p_1, \ldots, p_{j+1})$

$O(n \cdot h)$

Theorem. The convex hull $CH(P)$ of n points P in $\mathbb{R}^2$ can be computed in $O(n \cdot h)$ time using *Gift Wrapping* (also called *Jarvis' March*), where $h = |CH(P)|$.

Proof (sketch): show P is right of $\overrightarrow{p_i p_{i+1}}$ by induction on i

- point p_1 is on CH(P); assume $(p_1, \ldots, p_i)$ are first i points on $CH(P)$
- point p_{i+1} must be right of $\overrightarrow{p_{i-1} p_i}$
- by choice of p_{i+1} all other points are right of $\overrightarrow{p_i p_{i+1}}$

Comparison of the Algorithms

Which algorithm is better?

- Graham's Scan: $O(n \log n)$ time.
- Jarvis' March: $O(n \cdot h)$ time.

Comparison of the Algorithms

Which algorithm is better?

- Graham's Scan: $O(n \log n)$ time.
- Jarvis' March: $O(n \cdot h)$ time.

- Well, they are not comparable! It depends on how large $h = |CH(P)|$ is!

Comparison of the Algorithms

Which algorithm is better?

- Graham's Scan: $O(n \log n)$ time.
- Jarvis' March: $O(n \cdot h)$ time.

- Well, they are not comparable! It depends on how large $h = |CH(P)|$ is!

Ques: Can we combine the two approaches into an asymptotically optimal algorithm...?

Chan's Algorithm

Suppose we know h:

ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**

Compute with GrahamScan $CH(P_i)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** h **do**

for $i = 1$ **to** $\lceil n/h \rceil$ **do**

$q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

$p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:

ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**
  Compute with GrahamScan $CH(P_i)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P
$p_0 \leftarrow (x_1, \infty)$
for $j = 1$ **to** h **do**
  for $i = 1$ **to** $\lceil n/h \rceil$ **do**
    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$
return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:

ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**
  Compute with GrahamScan $CH(P_i)$

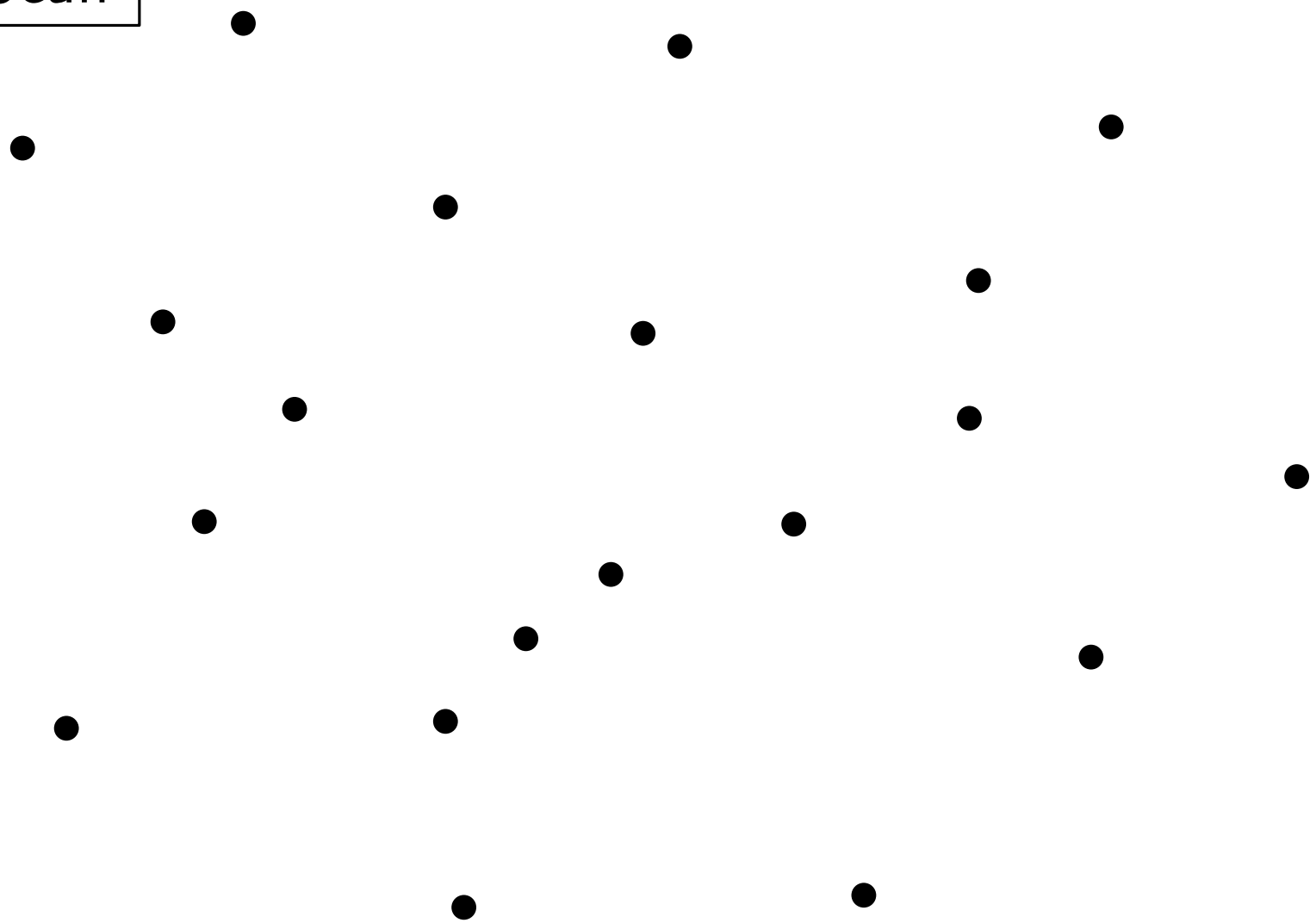
GrahamScan

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P
$p_0 \leftarrow (x_1, \infty)$
for $j = 1$ **to** h **do**
  for $i = 1$ **to** $\lceil n/h \rceil$ **do**
    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$
return $(p_1, \ldots, p_h)$

Gift Wrapping

Example

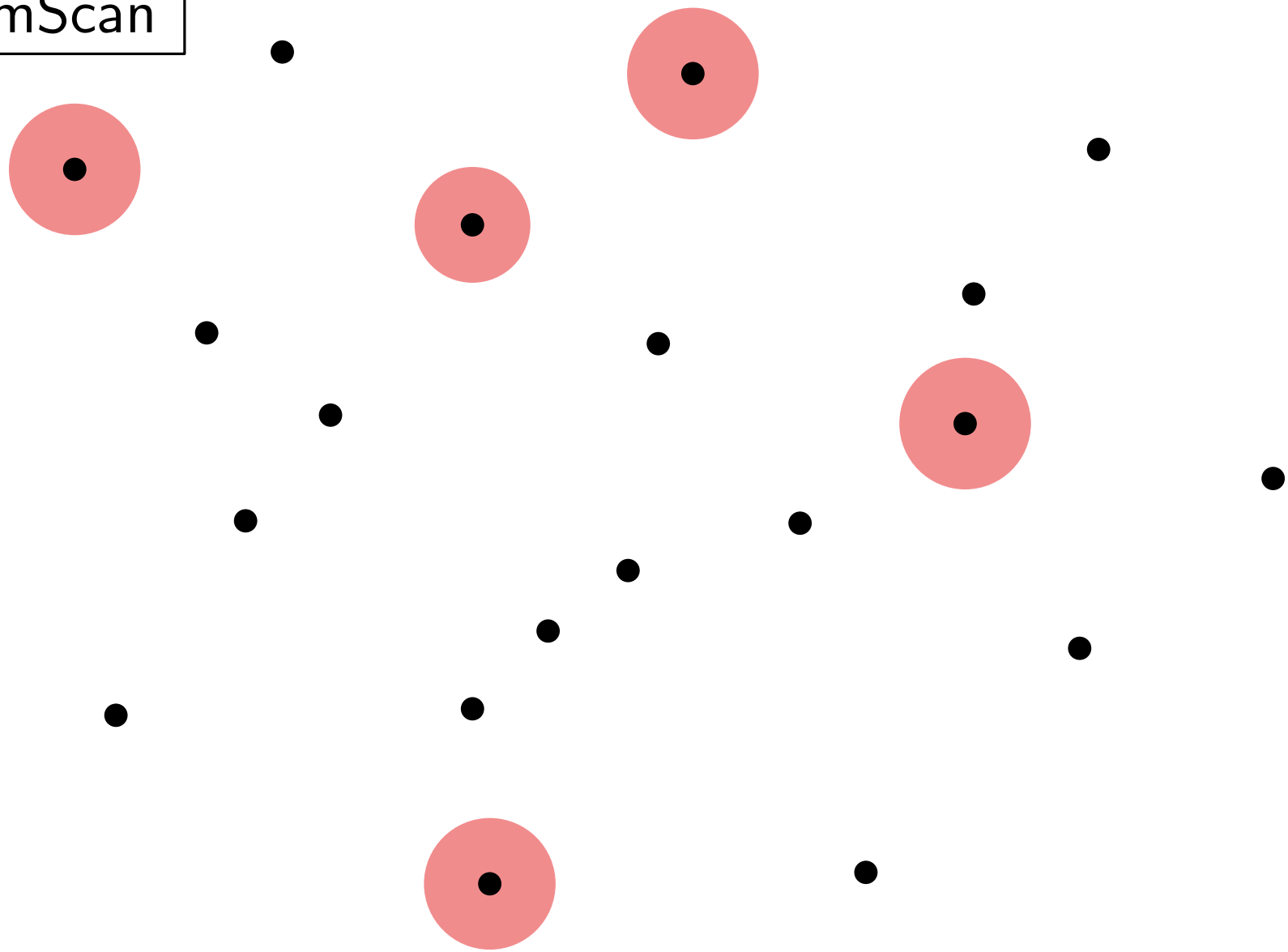
GrahamScan



n = 20

Example

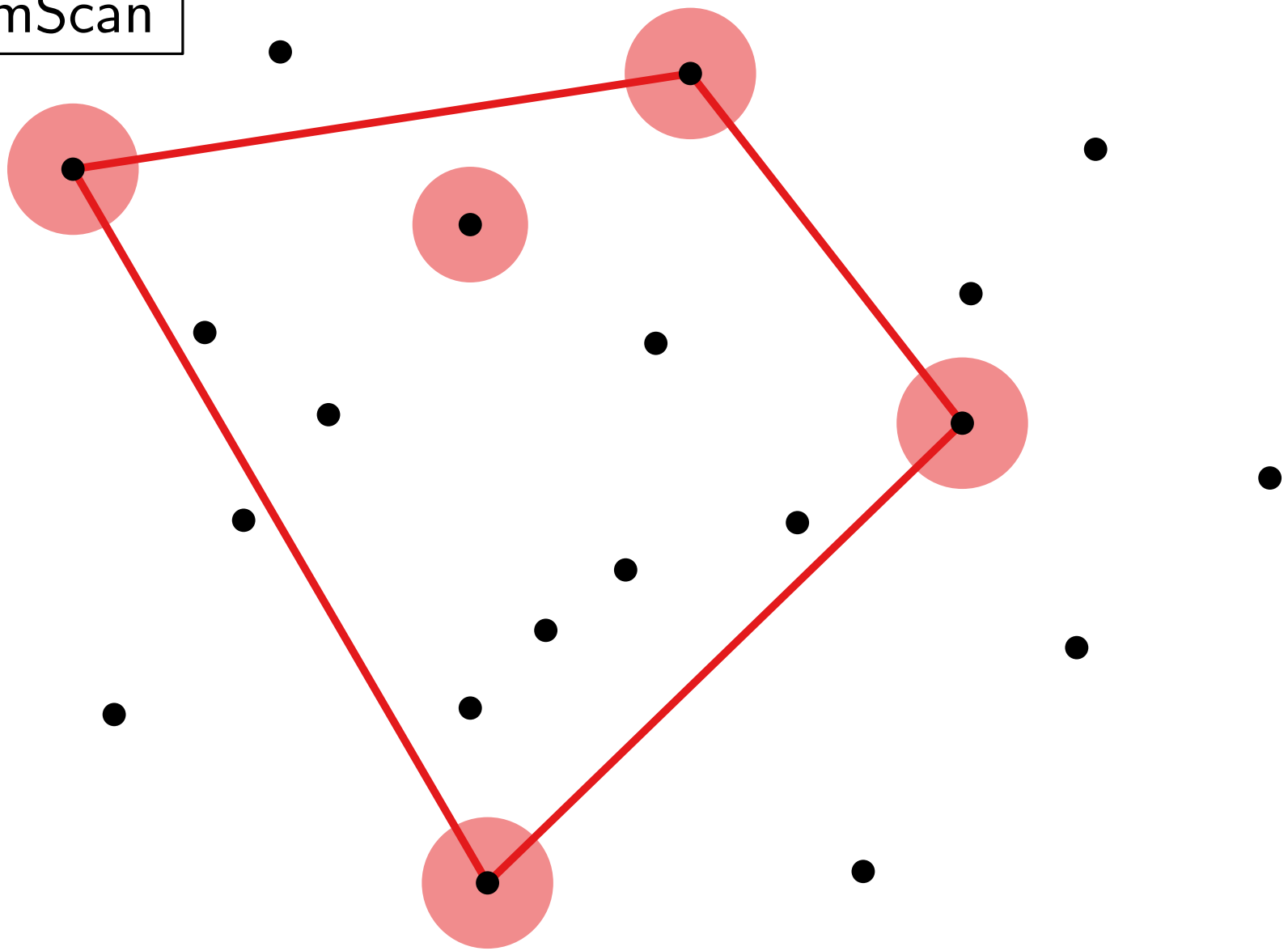
GrahamScan



n = 20

Example

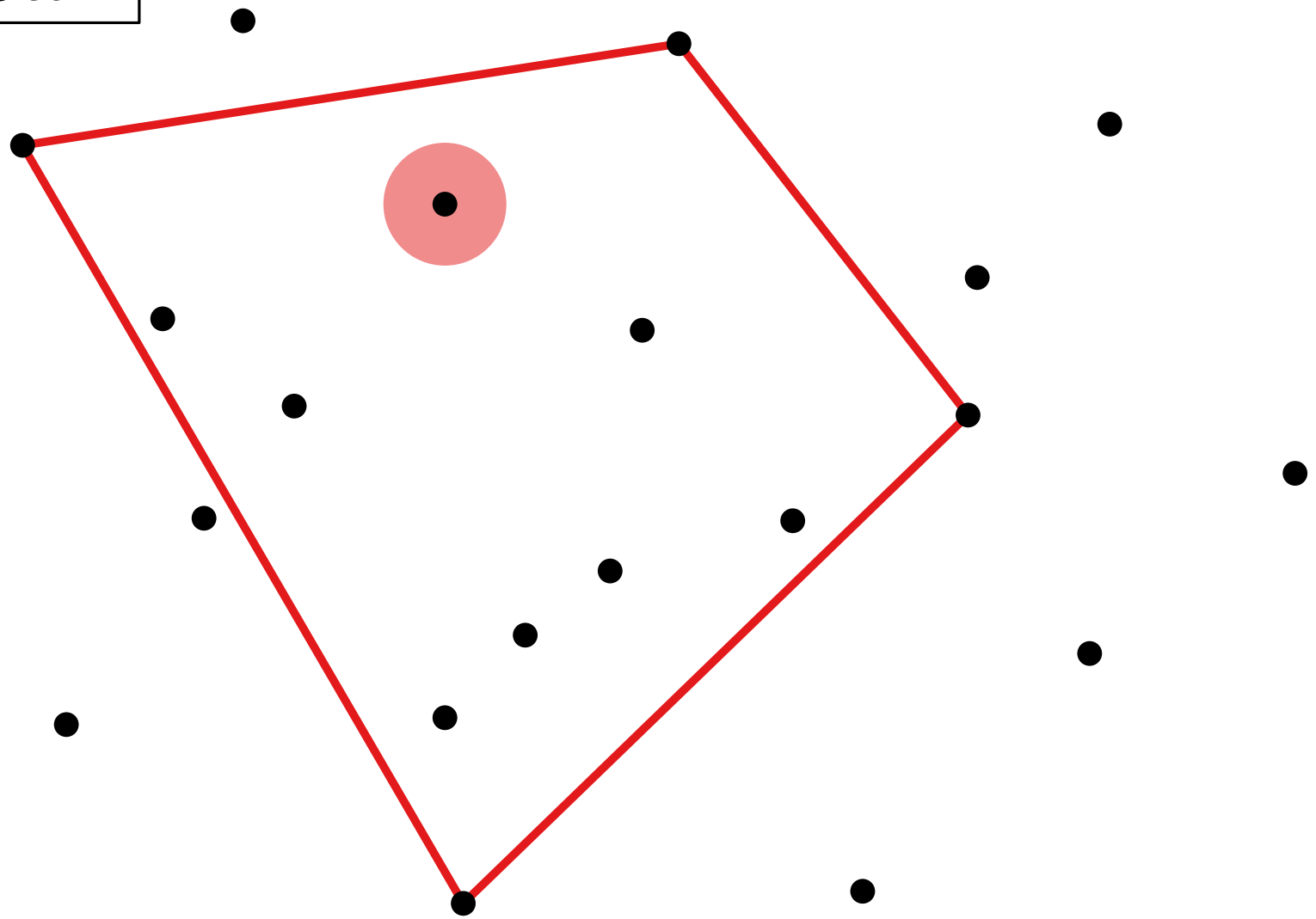
GrahamScan



$n = 20$

Example

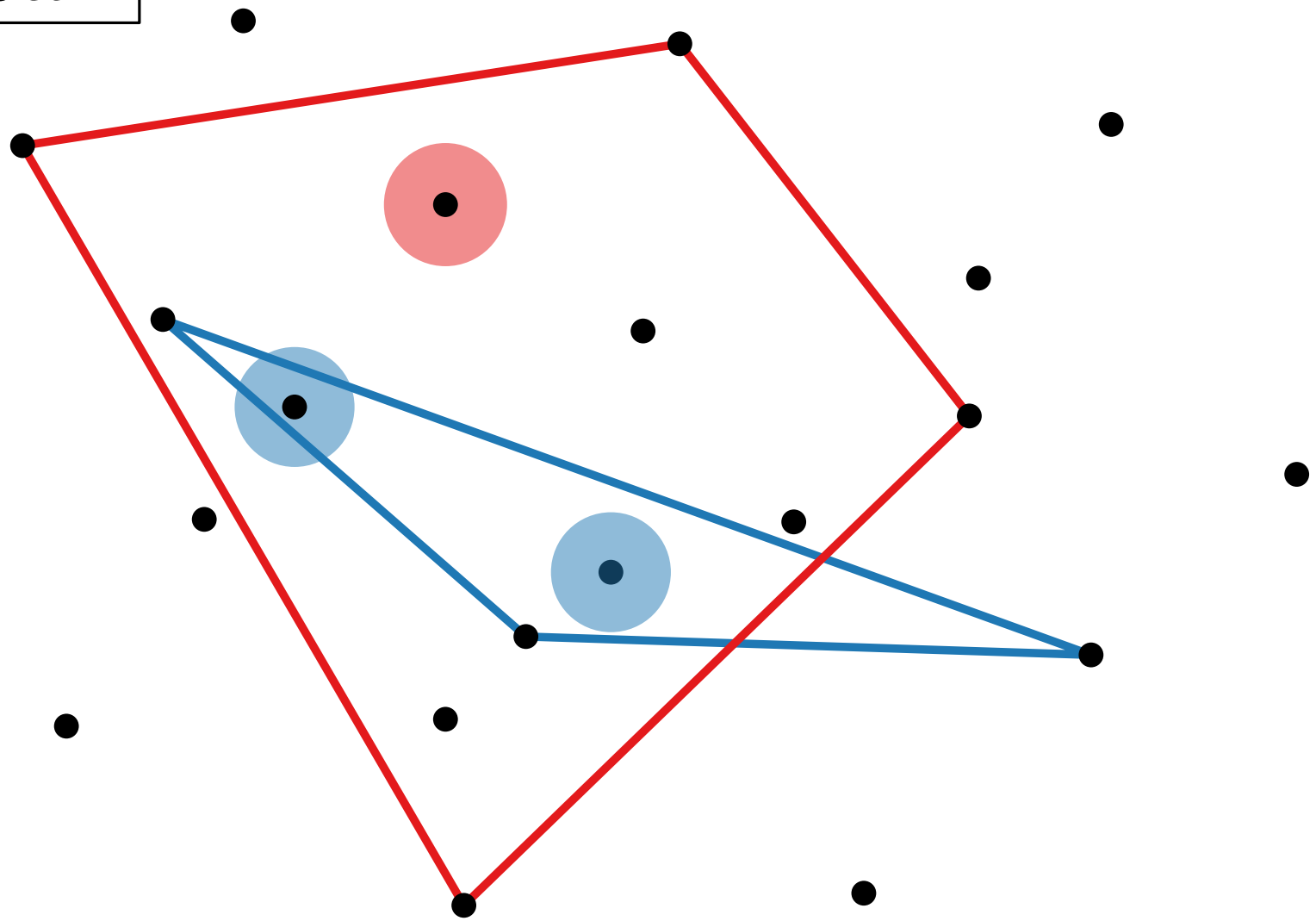
GrahamScan



$n = 20$

Example

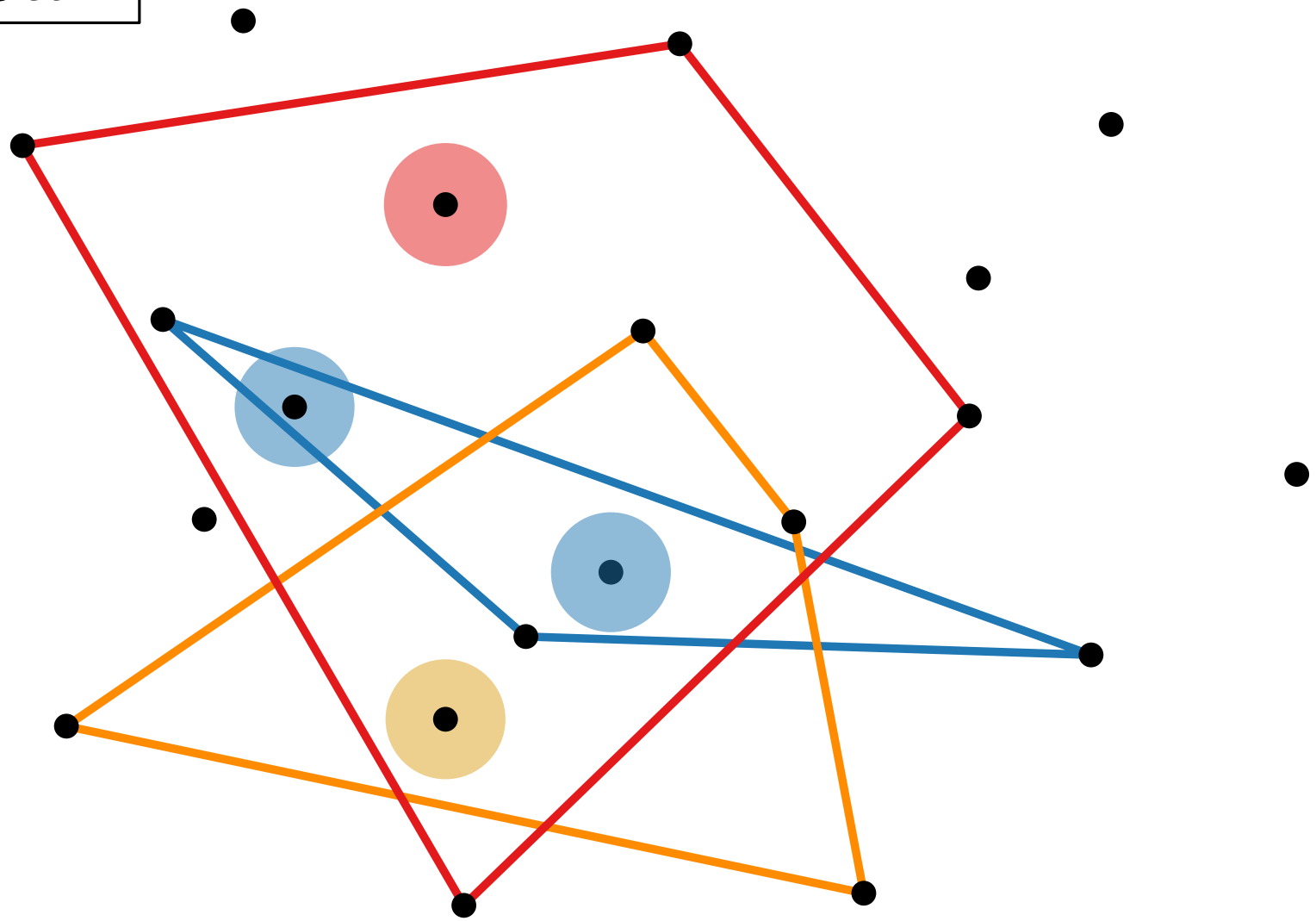
GrahamScan



n = 20

Example

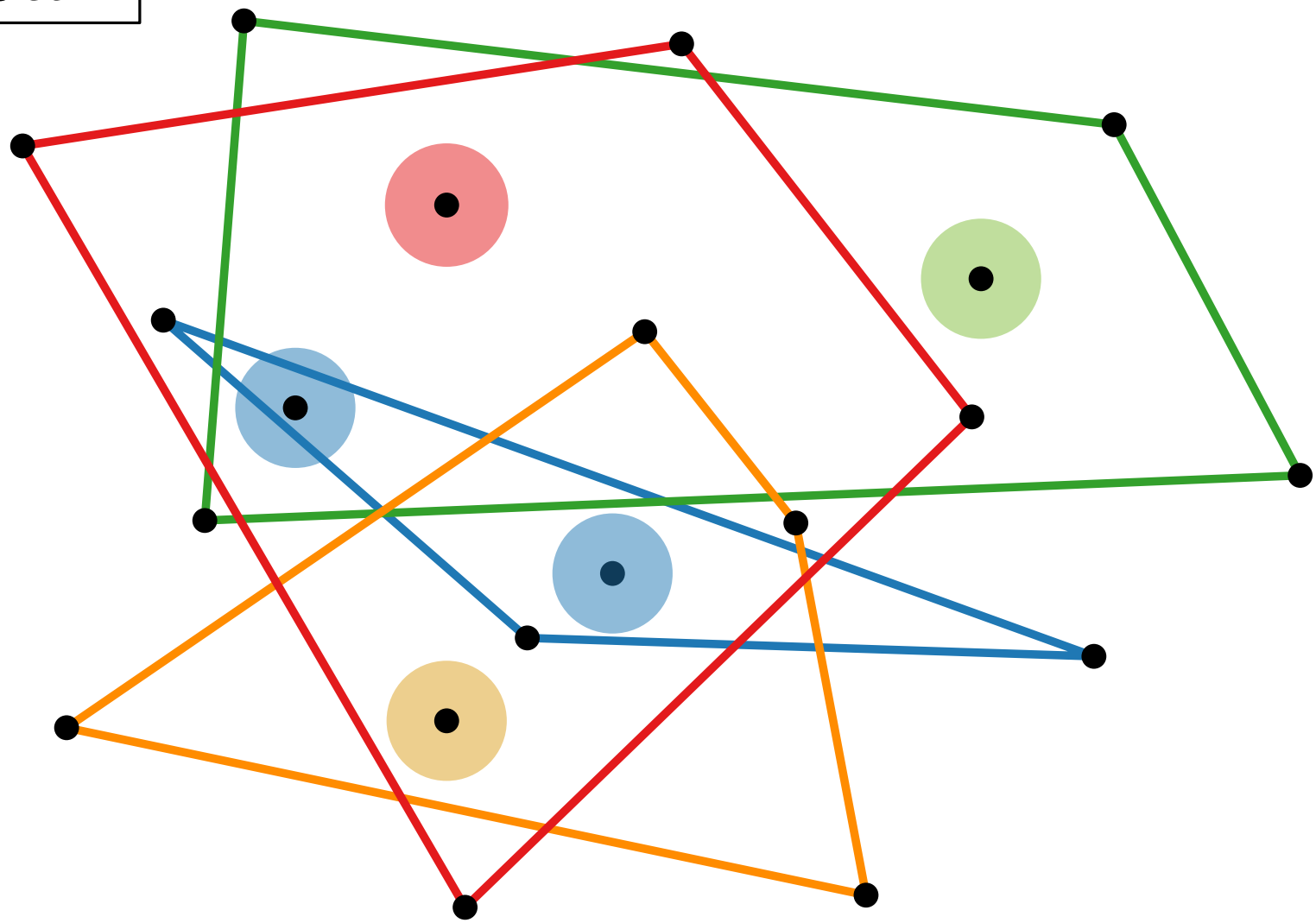
GrahamScan



n = 20

Example

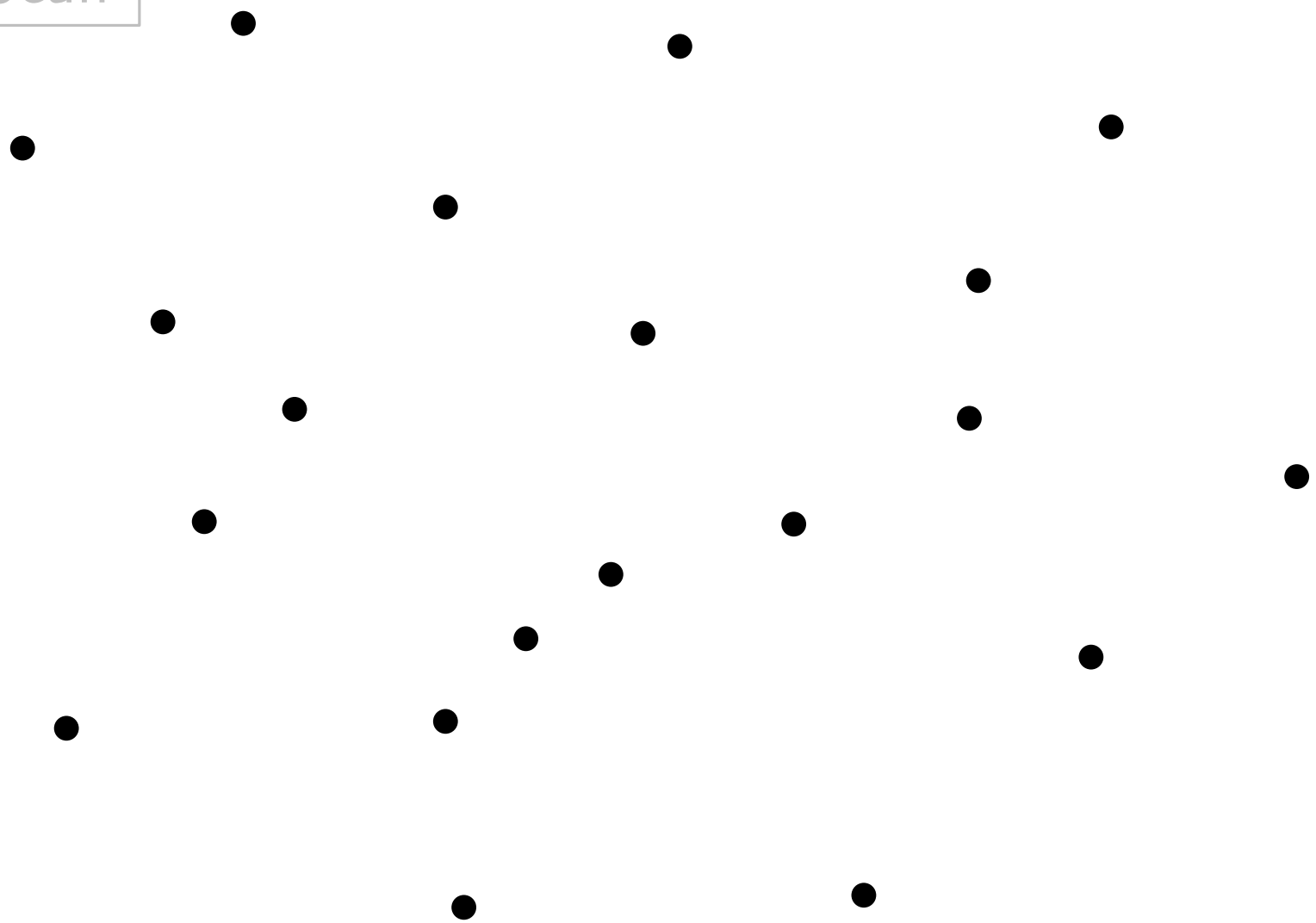
GrahamScan



n = 20

Example

GrahamScan

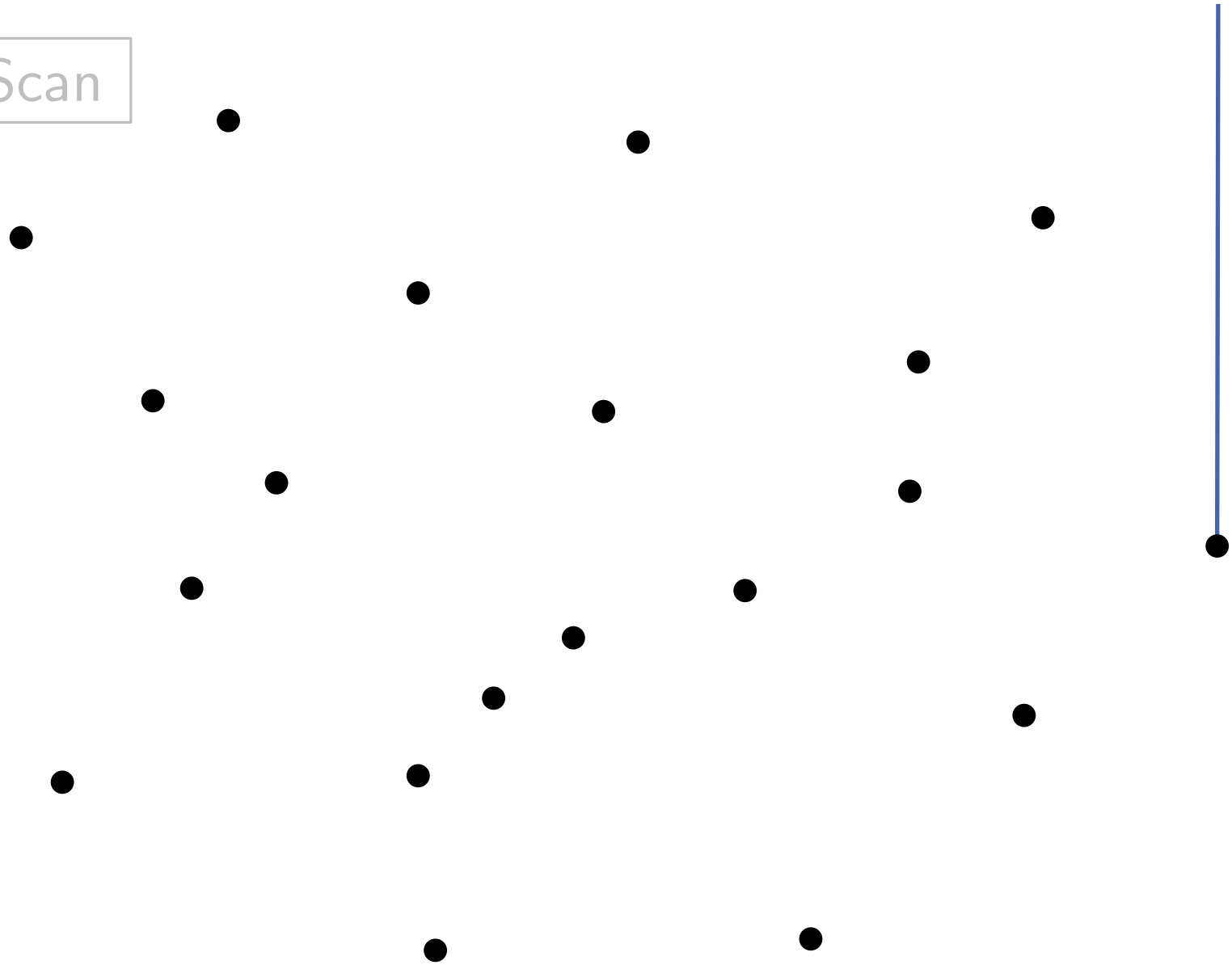


$n = 20$

Gift Wrapping

Example

GrahamScan

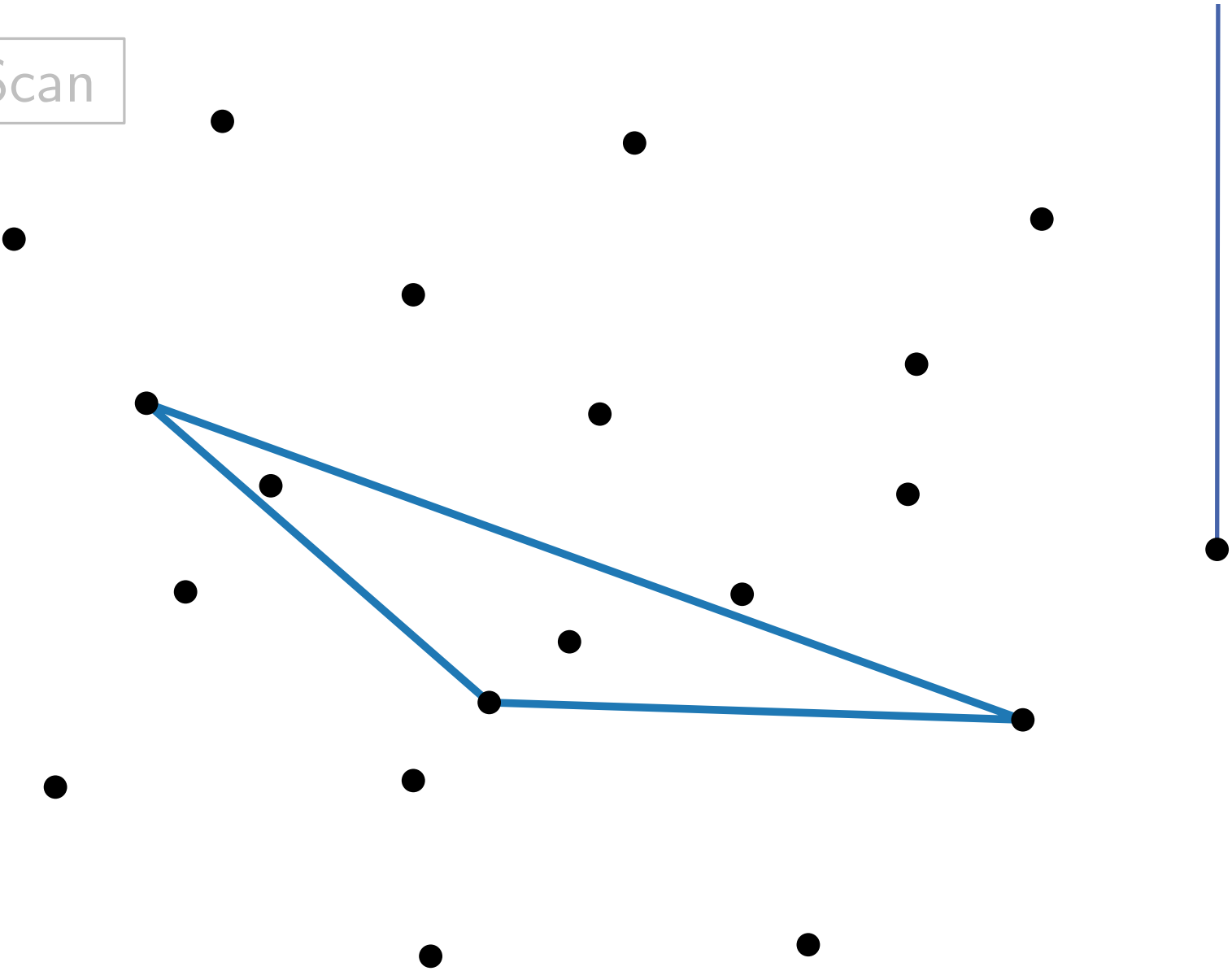


$n = 20$

Gift Wrapping

Example

GrahamScan

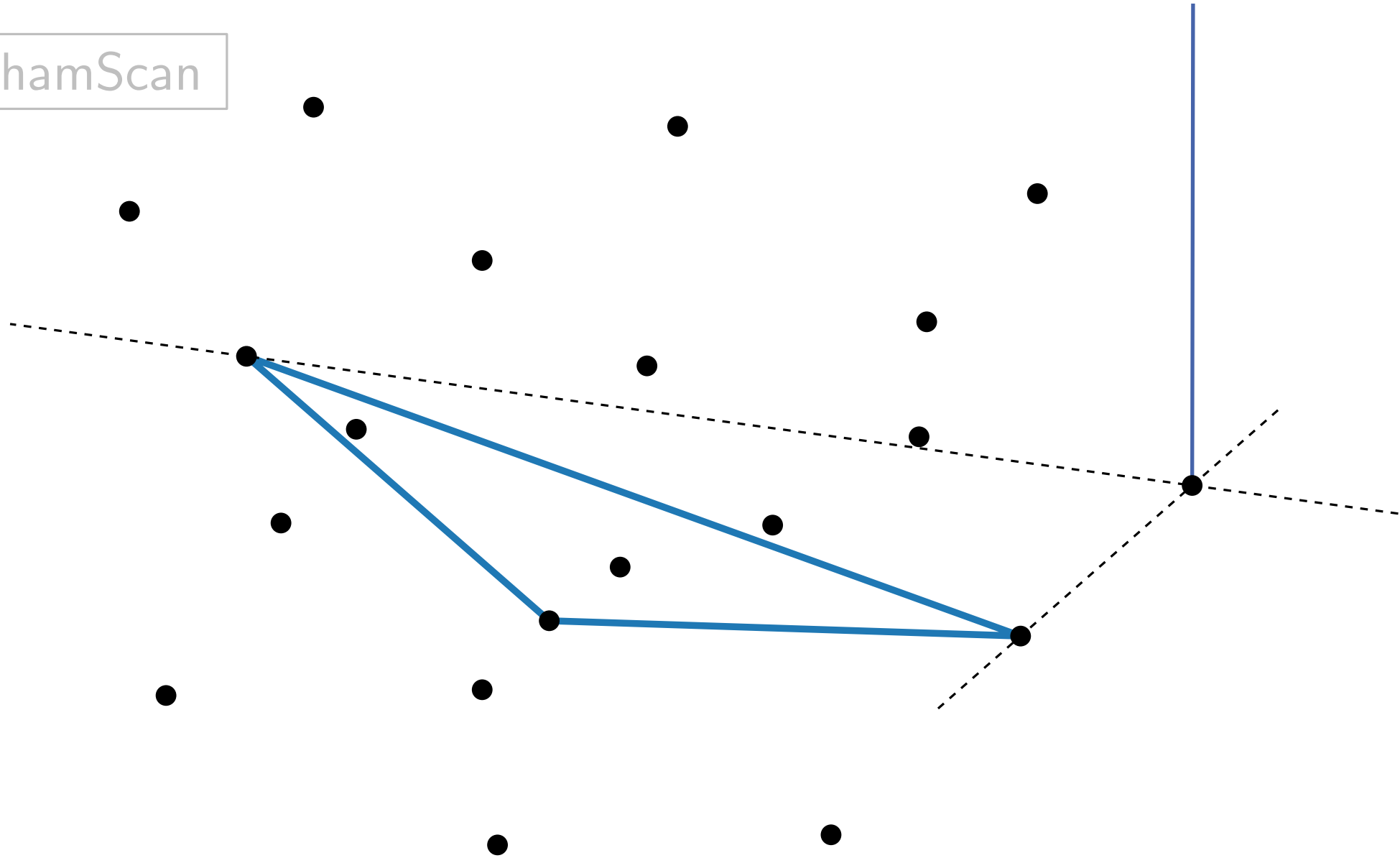


n = 20

Gift Wrapping

Example

GrahamScan

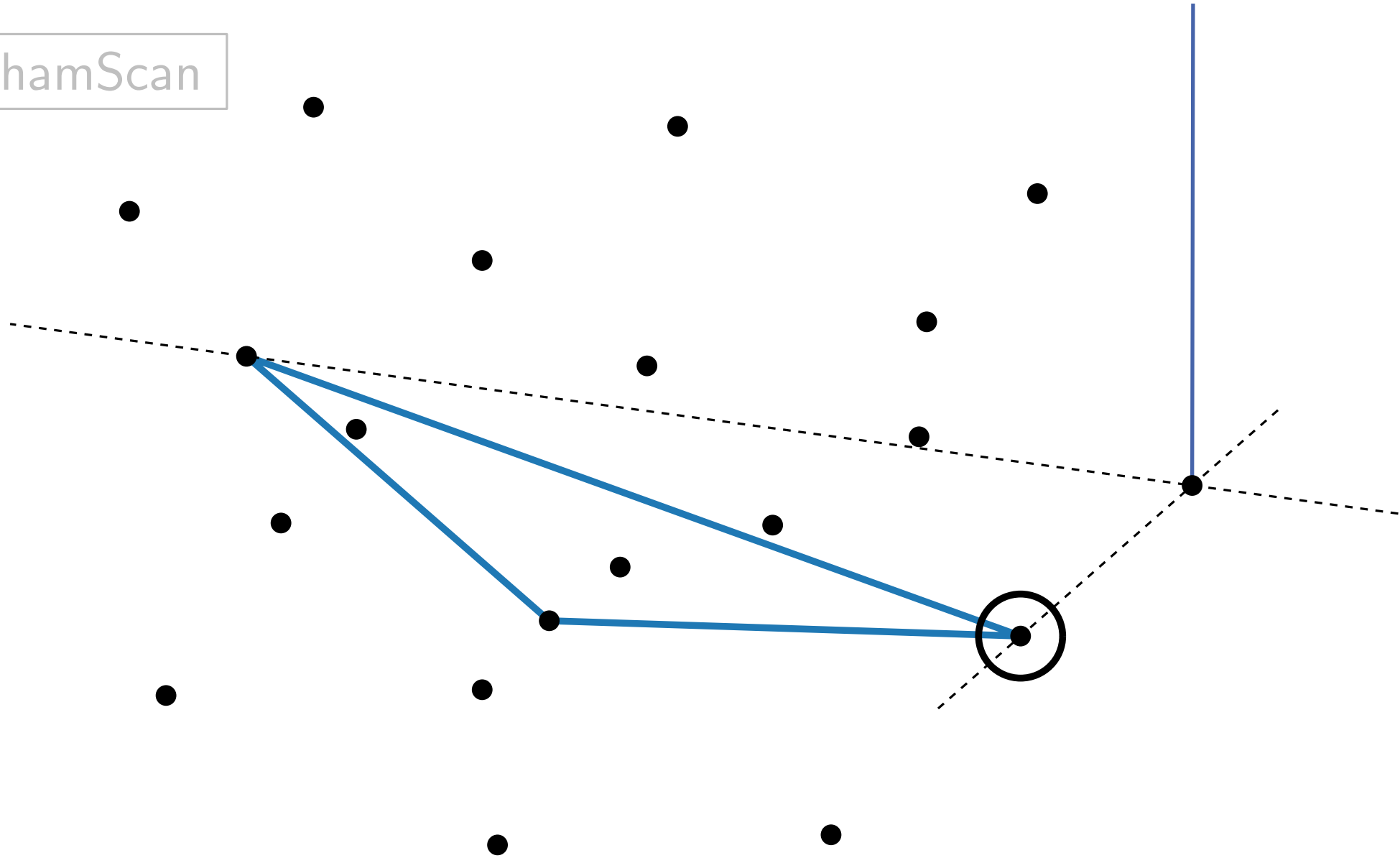


n = 20

Gift Wrapping

Example

GrahamScan

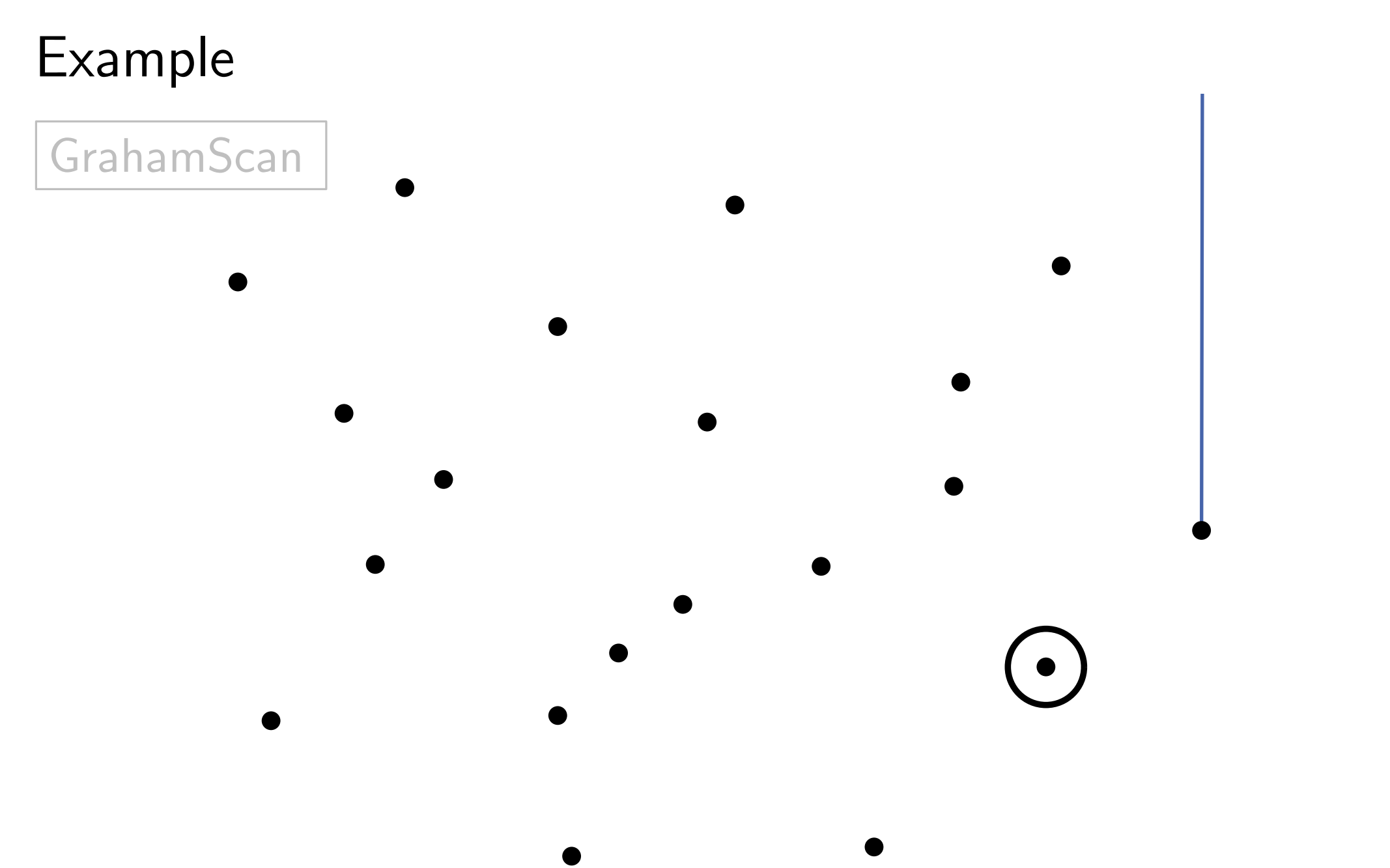


n = 20

Gift Wrapping

Example

GrahamScan

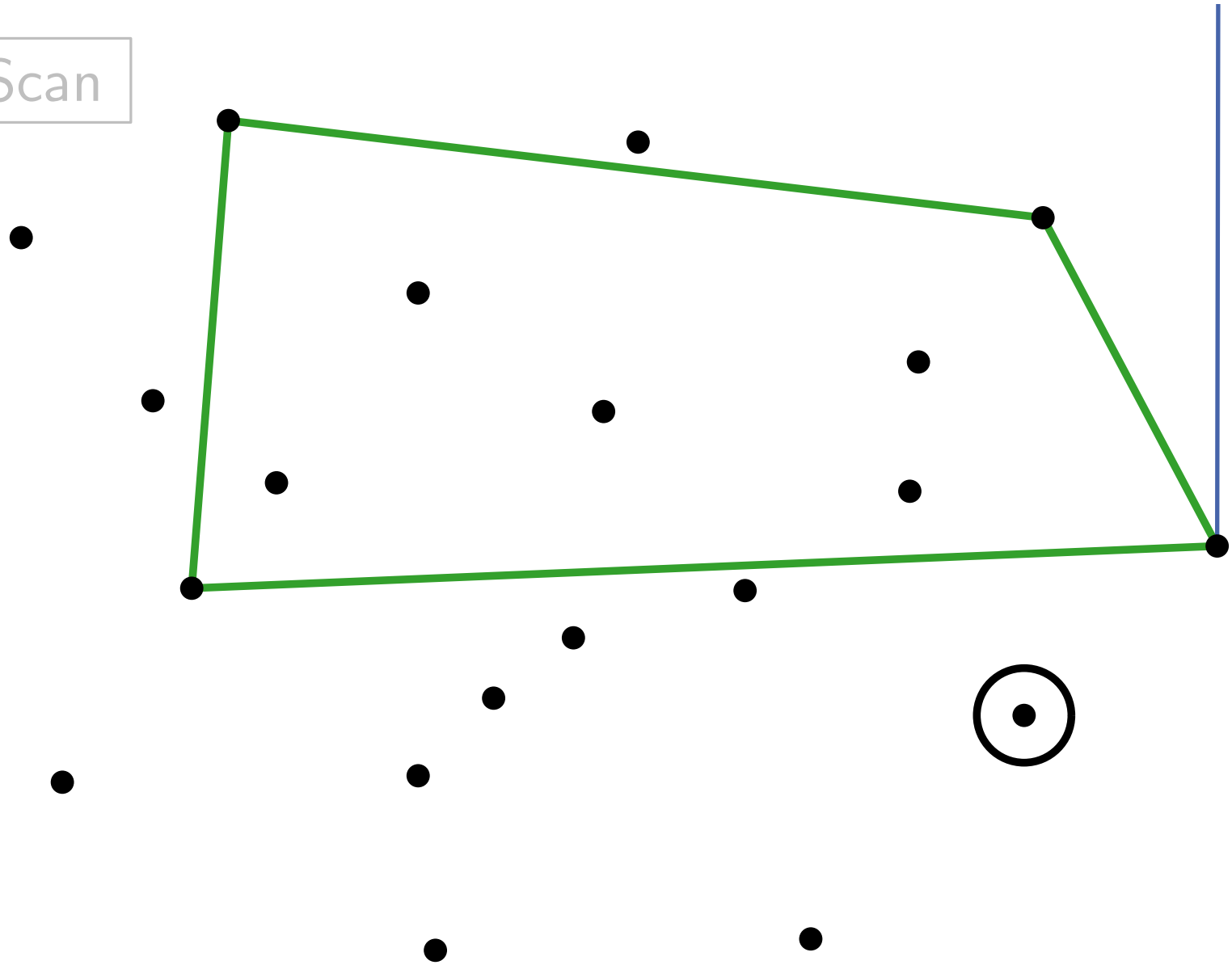


$n = 20$

Gift Wrapping

Example

GrahamScan

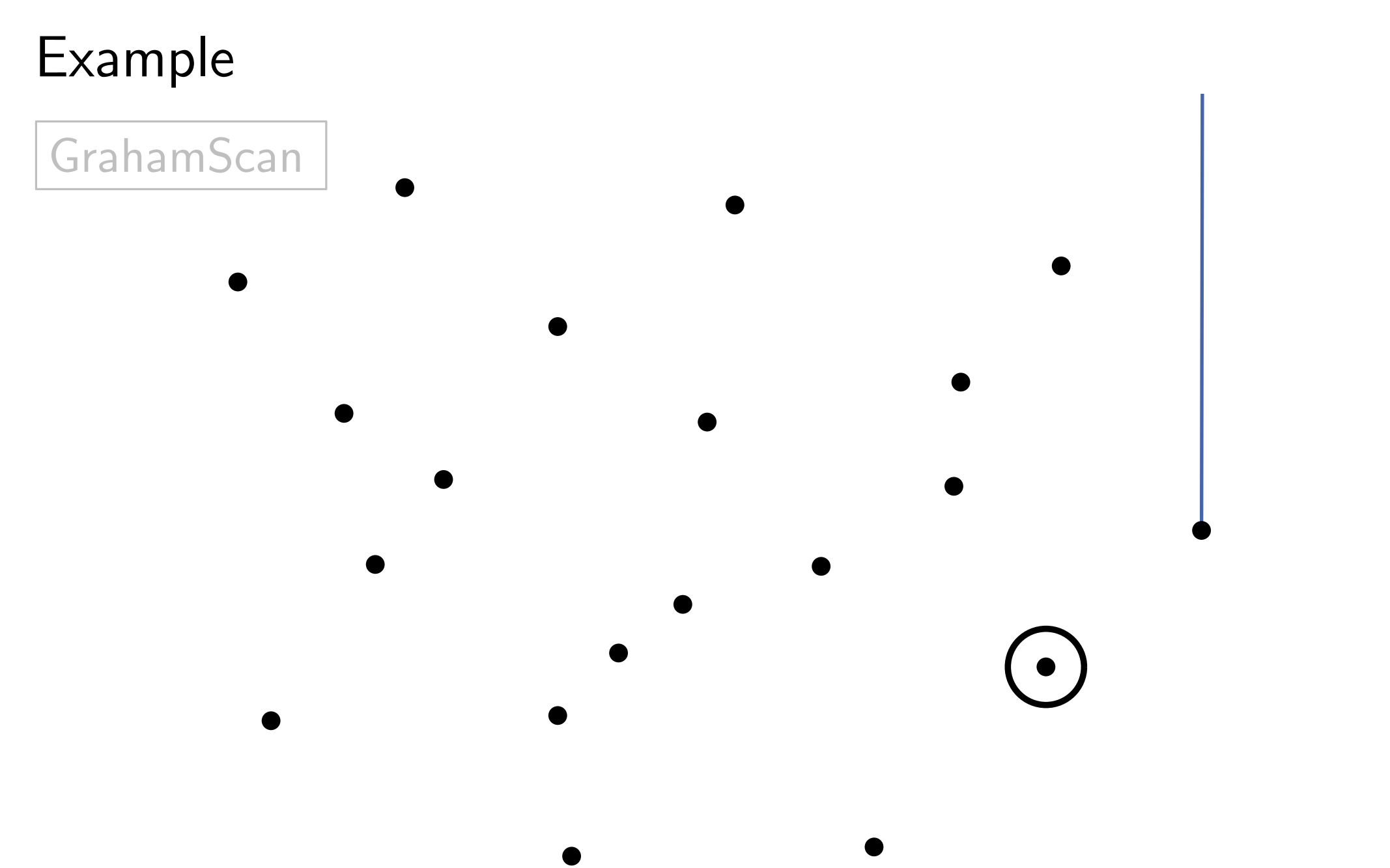


$n = 20$

Gift Wrapping

Example

GrahamScan

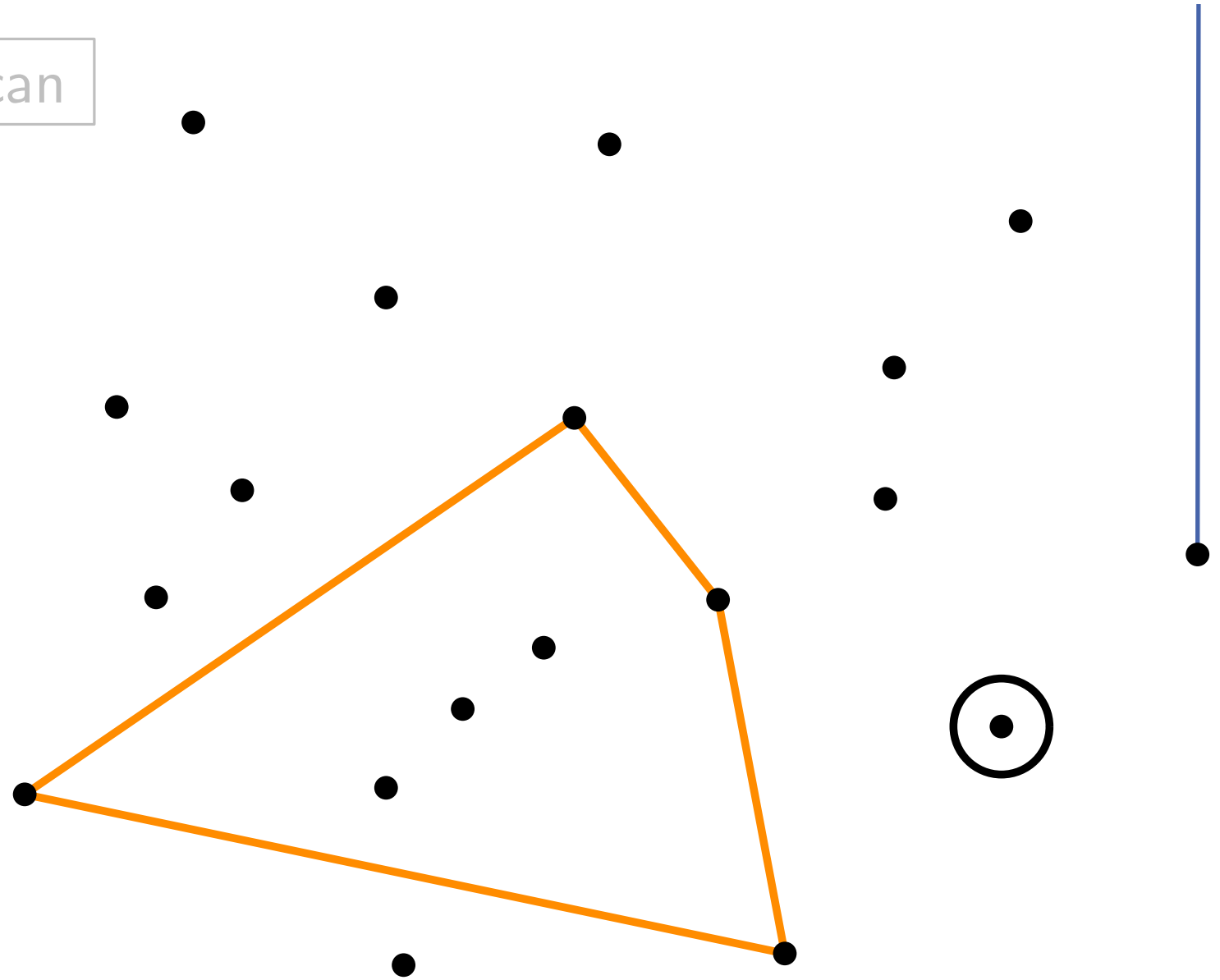


$n = 20$

Gift Wrapping

Example

GrahamScan

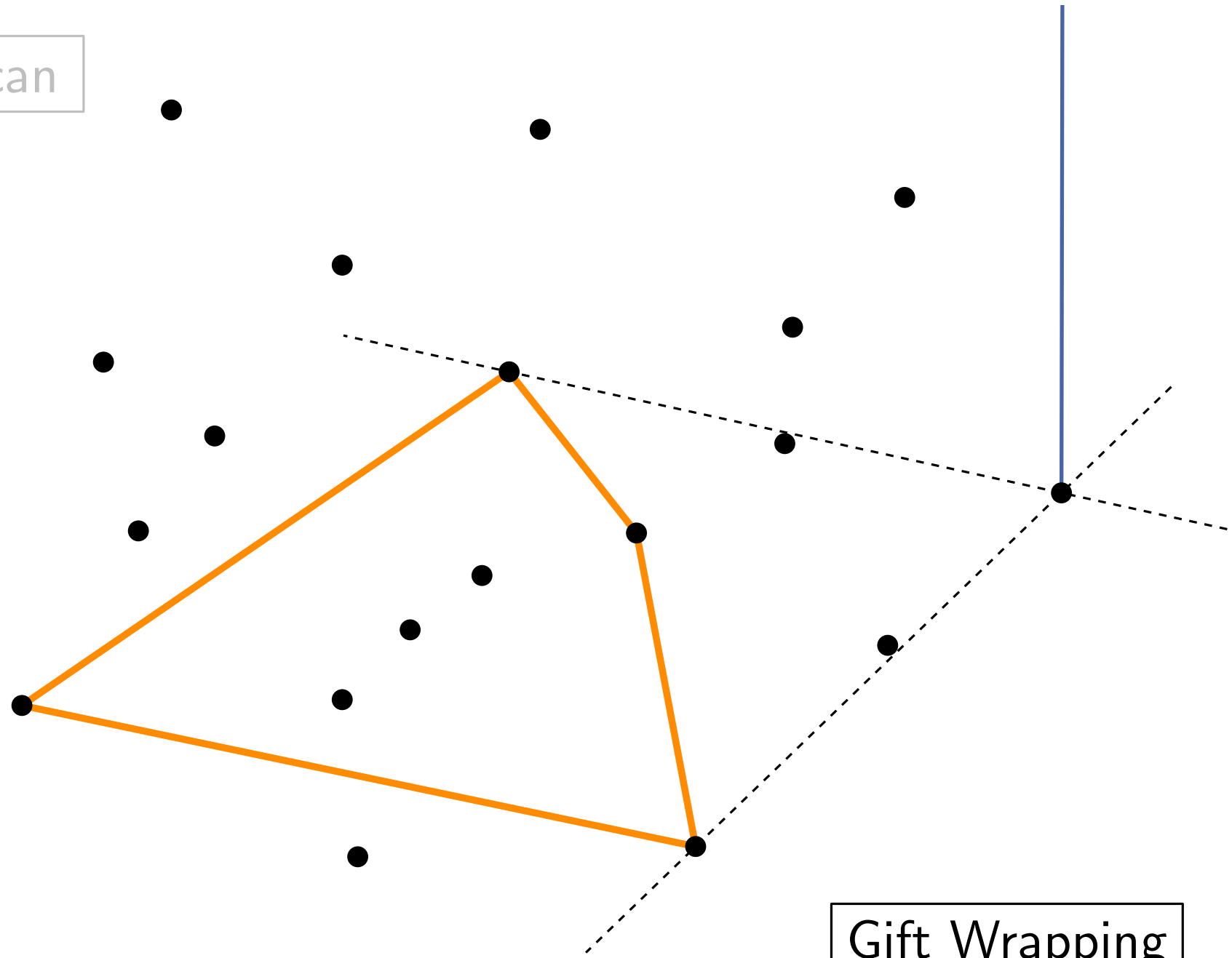


$n = 20$

Gift Wrapping

Example

GrahamScan

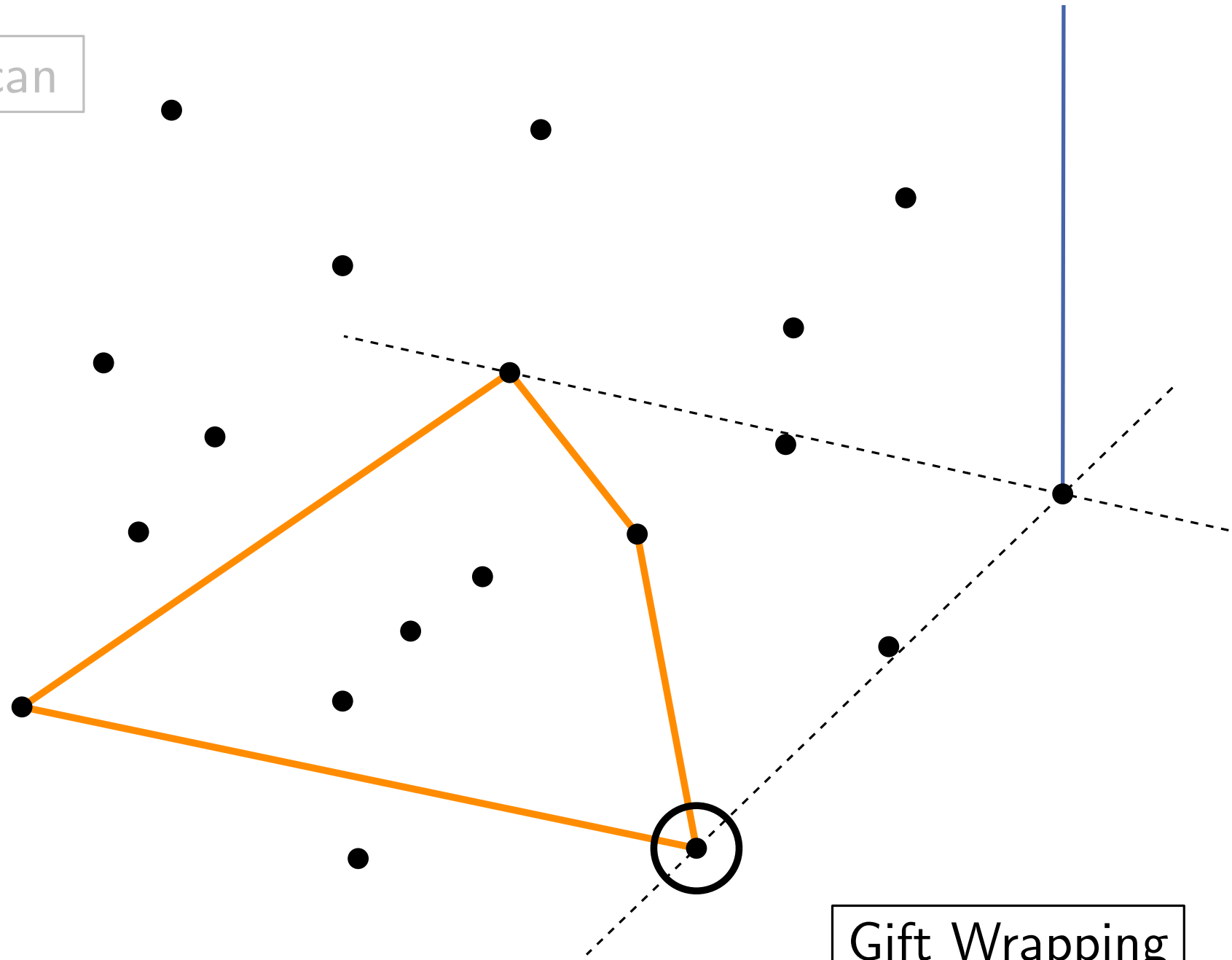


n = 20

Gift Wrapping

Example

GrahamScan

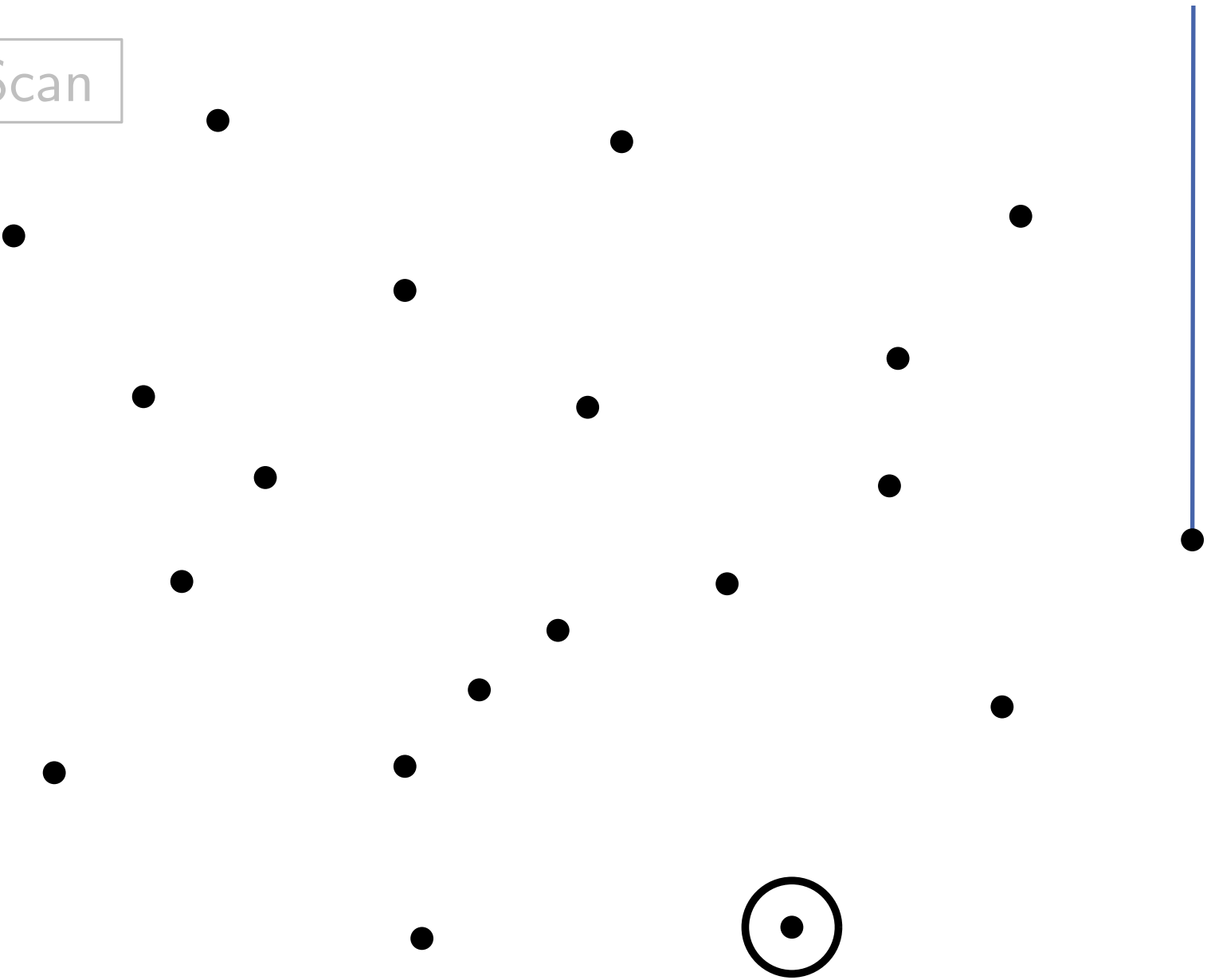


n = 20

Gift Wrapping

Example

GrahamScan

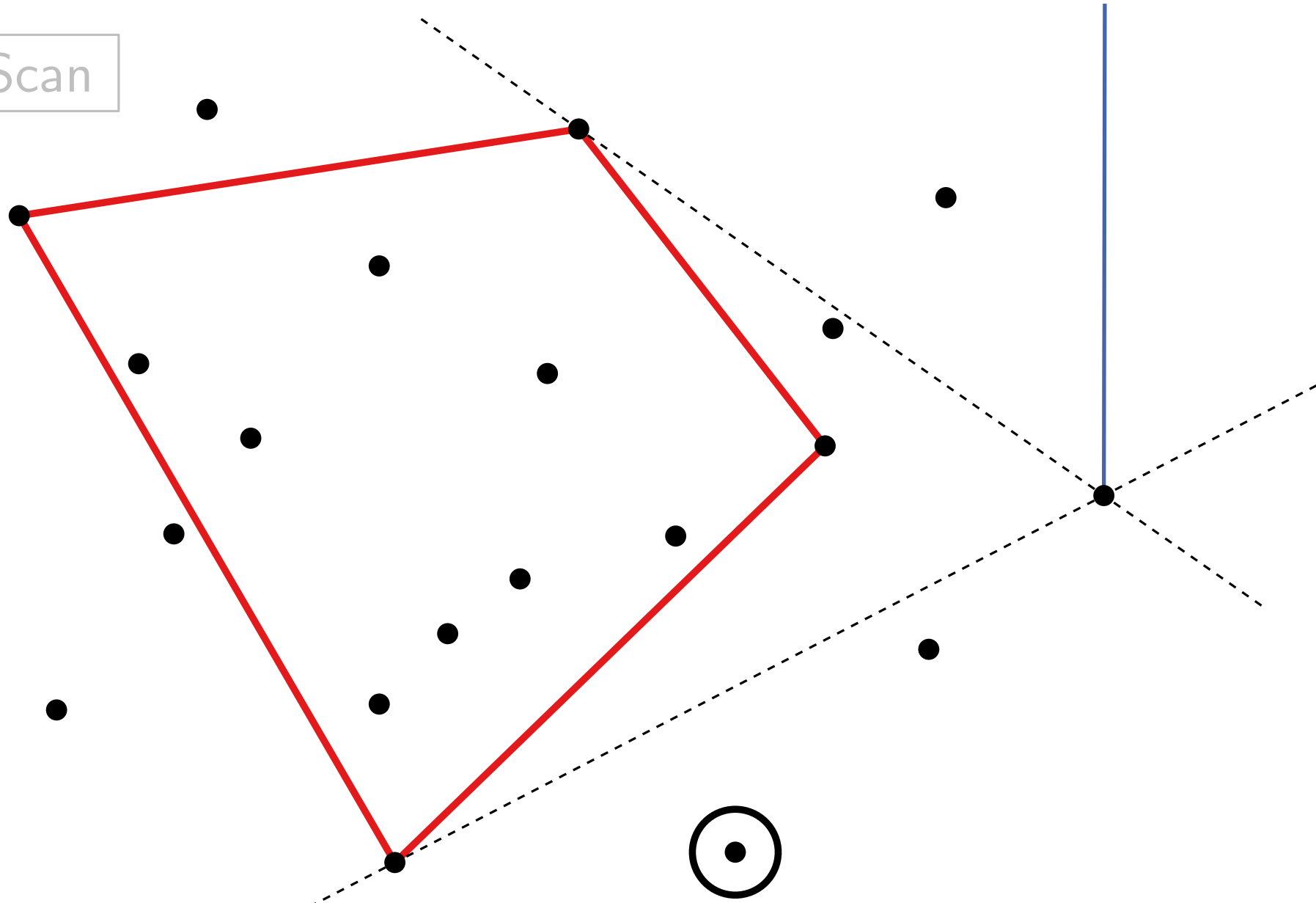


Gift Wrapping

n = 20

Example

GrahamScan

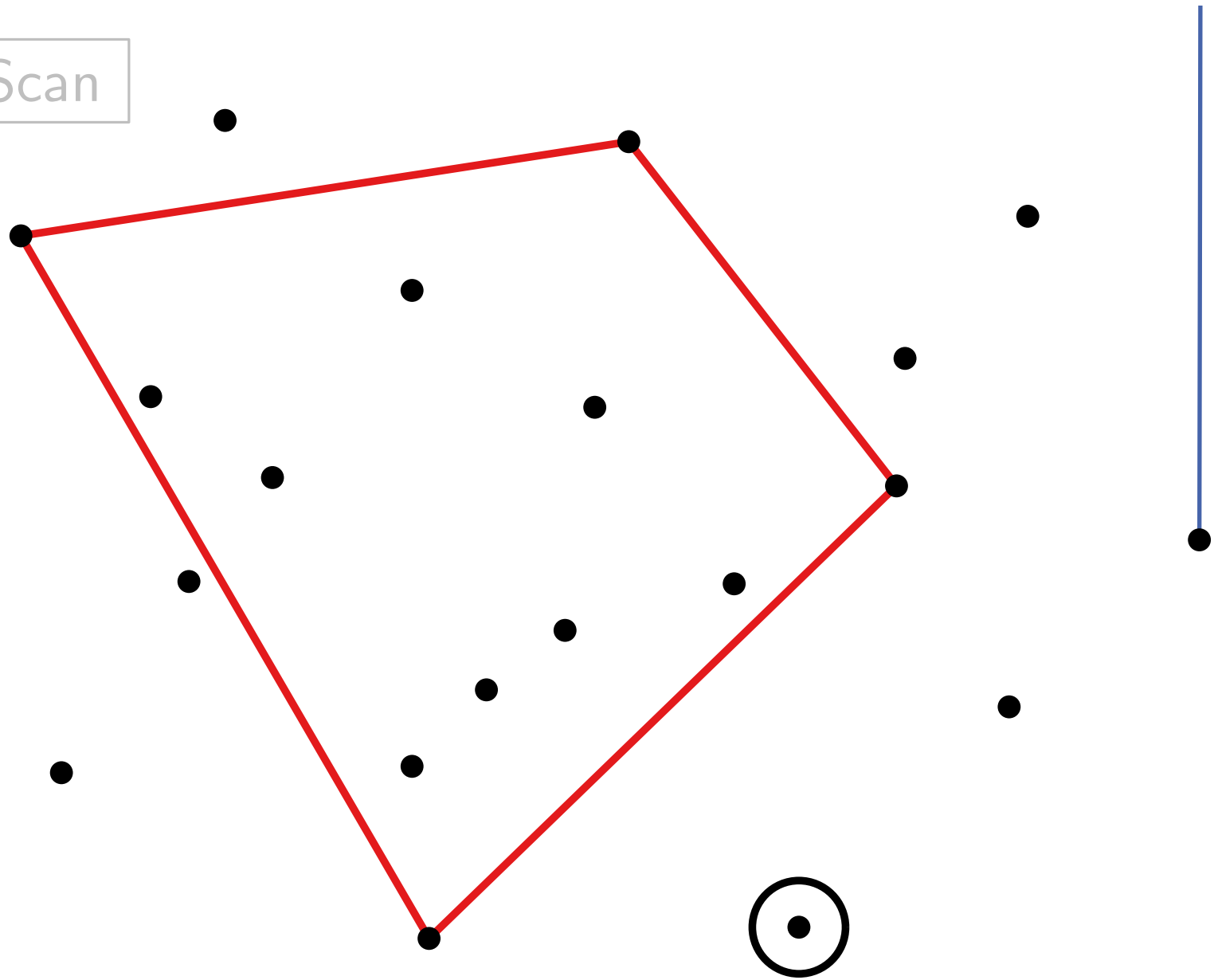


n = 20

Gift Wrapping

Example

GrahamScan

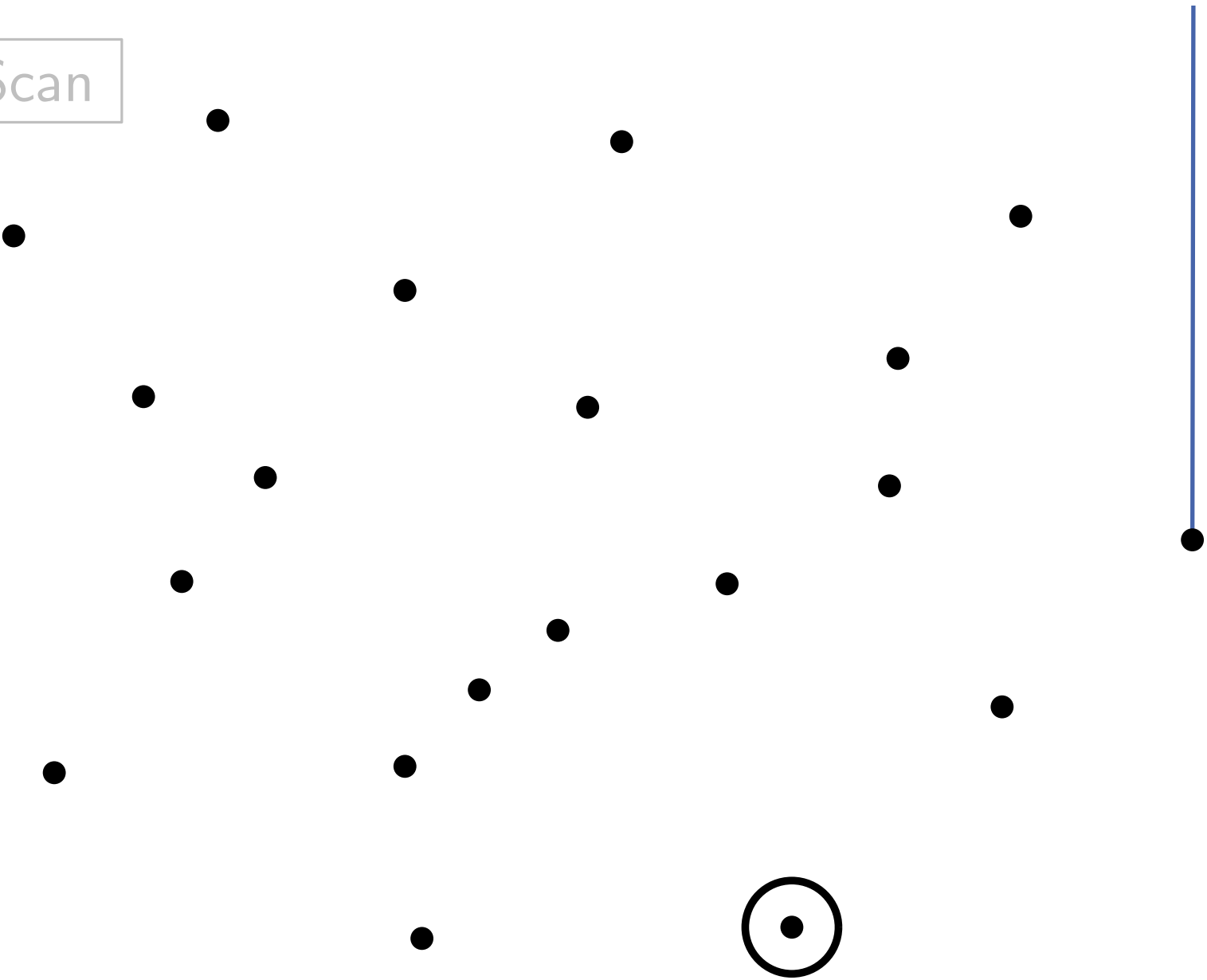


n = 20

Gift Wrapping

Example

GrahamScan

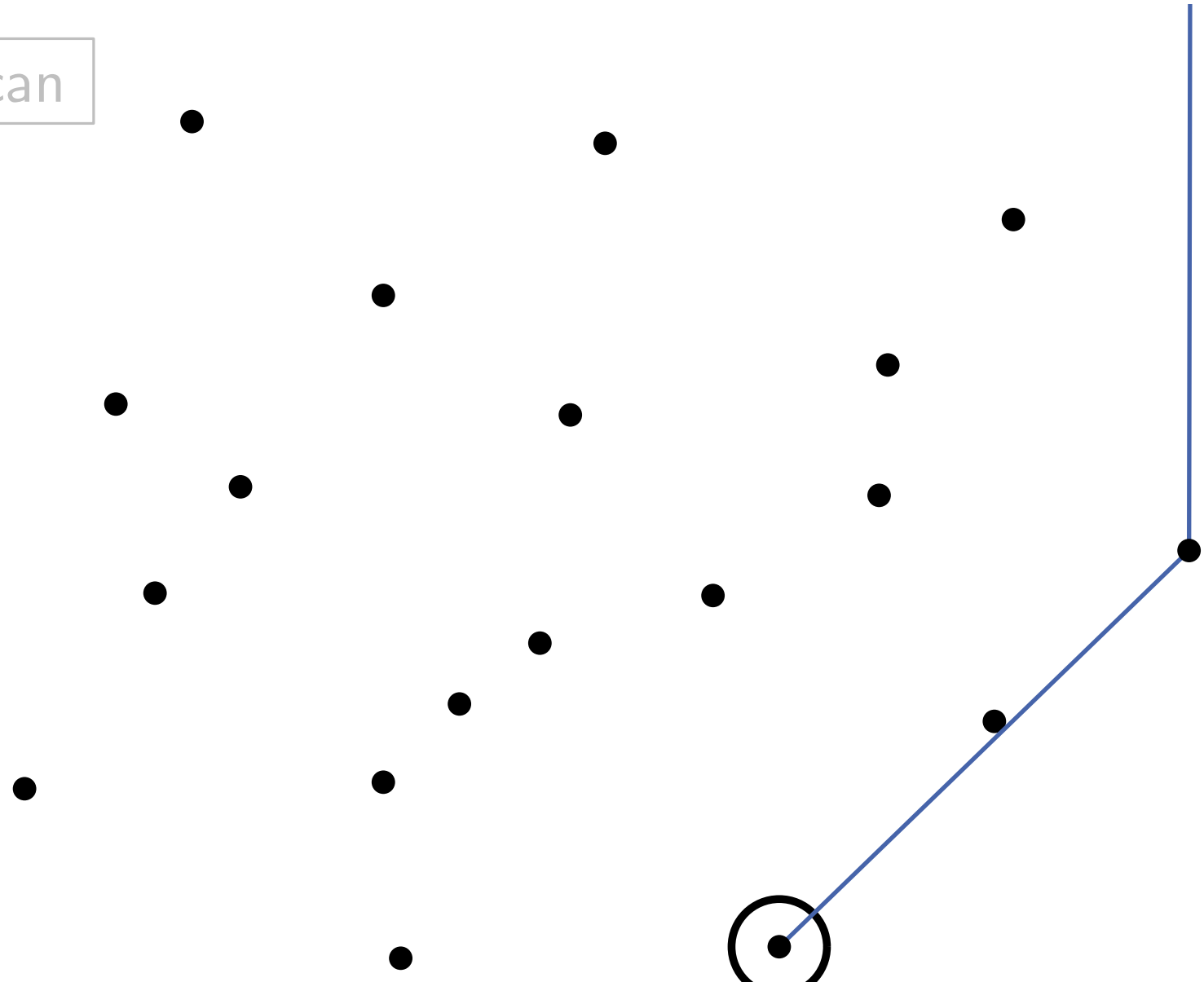


Gift Wrapping

n = 20

Example

GrahamScan



$n = 20$

Gift Wrapping

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**
  Compute with GrahamScan $CH(P_i)$

GrahamScan

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P
$p_0 \leftarrow (x_1, \infty)$
for $j = 1$ **to** h **do**
  for $i = 1$ **to** $\lceil n/h \rceil$ **do**
    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$
return $(p_1, \ldots, p_h)$

Gift Wrapping

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**
  Compute with GrahamScan $CH(P_i)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P
$p_0 \leftarrow (x_1, \infty)$
for $j = 1$ **to** h **do**
  for $i = 1$ **to** $\lceil n/h \rceil$ **do**
    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$
return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do**
  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P
$p_0 \leftarrow (x_1, \infty)$
for $j = 1$ **to** h **do**
  for $i = 1$ **to** $\lceil n/h \rceil$ **do**
    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$
  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$
return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do**

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do**

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do**

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

$\mathcal{O}(\log h) \rightarrow$ exercise-1

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

$\mathcal{O}(h) \cdot \mathcal{O}(n/h) = \mathcal{O}(n)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do** $\mathcal{O}(\log h) \rightarrow$ exercise-1

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

$\mathcal{O}(h) \cdot \mathcal{O}(n/h) = \mathcal{O}(n)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do** $\mathcal{O}(\log h) \rightarrow$ exercise-1

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

$\mathcal{O}(n \log h)$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

Suppose we know h:
ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

$\mathcal{O}(h) \cdot \mathcal{O}(n/h) = \mathcal{O}(n)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do** $\mathcal{O}(\log h) \rightarrow$ exercise-1

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

$\mathcal{O}(n \log h)$

return $(p_1, \ldots, p_h)$

Total: $\mathcal{O}(n \log h)$

Chan's Algorithm

But in general h is unknown!

ChanHull(P,h)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

$\mathcal{O}(h) \cdot \mathcal{O}(n/h) = \mathcal{O}(n)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do**

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

$\mathcal{O}(\log h) \rightarrow$ exercise-1

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

$\mathcal{O}(n \log h)$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

But in general h is unknown!

ChanHull(P, ~~h~~ m)

Divide P into sets P_i with $\leq h$ points

for i from 1 to $\lceil n/h \rceil$ **do** $\mathcal{O}(n/h)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(h \log h)$

$\mathcal{O}(n \log h)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

$\mathcal{O}(h) \cdot \mathcal{O}(n/h) = \mathcal{O}(n)$

for $j = 1$ **to** h **do**

  for $i = 1$ **to** $\lceil n/h \rceil$ **do** $\mathcal{O}(\log h) \rightarrow$ exercise-1

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/h \rceil}\}\}$

$\mathcal{O}(n \log h)$

return $(p_1, \ldots, p_h)$

Chan's Algorithm

But in general h is unknown!

ChanHull(P,m)

Divide P into sets P_i with $\leq m$ points

for i from 1 to $\lceil n/m \rceil$ **do** $\mathcal{O}(n/m)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(m \log m)$

$\mathcal{O}(n \log m)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** m **do**

$\mathcal{O}(m) \cdot \mathcal{O}(n/m) = \mathcal{O}(n)$

  for $i = 1$ **to** $\lceil n/m \rceil$ **do**

$\mathcal{O}(\log m)$

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/m \rceil}\}\}$

$\mathcal{O}(n \log m)$

return $(p_1, \ldots, p_m)$

Total: $\mathcal{O}(n \log m)$

Chan's Algorithm

But in general h is unknown!

ChanHull(P,m)

Divide P into sets P_i with $\leq m$ points

for i from 1 to $\lceil n/m \rceil$ **do** $\mathcal{O}(n/m)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(m \log m)$

$\mathcal{O}(n \log m)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** m **do**

$\mathcal{O}(m) \cdot \mathcal{O}(n/m) = \mathcal{O}(n)$

  for $i = 1$ **to** $\lceil n/m \rceil$ **do**

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$ $\mathcal{O}(\log m)$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/m \rceil}\}\}$

$\mathcal{O}(n \log m)$

return $(p_1, \ldots, p_m)$

Total: $\mathcal{O}(n \log m)$

Chan's Algorithm

But in general h is unknown!

ChanHull(P,m)

Divide P into sets P_i with $\leq m$ points

for i from 1 to $\lceil n/m \rceil$ **do** $\mathcal{O}(n/m)$

  Compute with GrahamScan $CH(P_i)$ $\mathcal{O}(m \log m)$

$\mathcal{O}(n \log m)$

$p_1 = (x_1, y_1) \leftarrow$ rightmost point in P

$p_0 \leftarrow (x_1, \infty)$

for $j = 1$ **to** m **do**

$\mathcal{O}(m) \cdot \mathcal{O}(n/m) = \mathcal{O}(n)$

  for $i = 1$ **to** $\lceil n/m \rceil$ **do**

$\mathcal{O}(\log m)$

    $q_i \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in P_i \setminus \{p_{j-1}, p_j\}\}$

  $p_{j+1} \leftarrow \arg\max\{\angle p_{j-1} p_j q \mid q \in \{q_1, \ldots, q_{\lceil n/m \rceil}\}\}$

  if $p_{j+1} = p_1$ **then return** $(p_1, \ldots, p_{j+1})$

$\mathcal{O}(n \log m)$

return failure

Total: $\mathcal{O}(n \log m)$

What to do with m?

Suggestions ...?

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

$m \leftarrow \min\{n, 2^{2^t}\}$

result $\leftarrow$ ChanHull(P, m)

if result $\neq$ failure **then** break

return result

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

$$\sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(n \log 2^{2^t})$$

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

$$\sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(n \log 2^{2^t}) = \mathcal{O}(n) \sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(2^t)$$

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

$$\sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(n \log 2^{2^t}) = \mathcal{O}(n) \sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(2^t)$$

$$\leq \mathcal{O}(n) \cdot \mathcal{O}(2^{\log \log h})$$

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

$$\sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(n \log 2^{2^t}) = \mathcal{O}(n) \sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(2^t)$$

$$\leq \mathcal{O}(n) \cdot \mathcal{O}(2^{\log \log h})$$

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m) $\mathcal{O}(n \log m) = \mathcal{O}(n \log 2^{2^t})$

  if result $\neq$ failure **then** break

return result

Running time:

$$\sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(n \log 2^{2^t}) = \mathcal{O}(n) \sum_{t=0}^{\lceil \log \log h \rceil} \mathcal{O}(2^t)$$

$$\leq \mathcal{O}(n) \cdot \mathcal{O}(2^{\log \log h}) = \mathcal{O}(n) \cdot \mathcal{O}(\log h)$$

$$= \mathcal{O}(n \log h)$$

What to do with m?

FullChanHull(P)

for $t = 0, 1, 2, \ldots$ **do**

  $m \leftarrow \min\{n, 2^{2^t}\}$

  result $\leftarrow$ ChanHull(P, m)

  if result $\neq$ failure **then** break

return result

Theorem. The convex hull $CH(P)$ of n points P in $\mathbb{R}^2$ can be computed in $O(n \log h)$ time with Chan's Algorithm, where $h = |CH(P)|$.

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Use lexicographic order! (if multiple points with same x-coordinates, then use y-coordinates).

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Use lexicographic order! (if multiple points with same x-coordinates, then use y-coordinates).

What happens with collinear points in $CH(P)$?

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Use lexicographic order! (if multiple points with same x-coordinates, then use y-coordinates).

What happens with collinear points in $CH(P)$?

Graham: Forms no right turn, so an interior point is deleted.
Jarvis: Choose farthest point

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Use lexicographic order! (if multiple points with same x-coordinates, then use y-coordinates).

What happens with collinear points in $CH(P)$?

Graham: Forms no right turn, so an interior point is deleted.
Jarvis: Choose farthest point

What about the robustness of the algorithms?

Discussion

Is it possible to be faster than $O(n \log n)$ or $O(n \log h)$ time?

Generally not! An algorithm to compute the convex hull can also sort n numbers (exercise!) $\Rightarrow$ lower bound $\Omega(n \log n)$.

What happens in Graham's Scan when sorting P is not unique?

Use lexicographic order! (if multiple points with same x-coordinates, then use y-coordinates).

What happens with collinear points in $CH(P)$?

Graham: Forms no right turn, so an interior point is deleted.
Jarvis: Choose farthest point

What about the robustness of the algorithms?

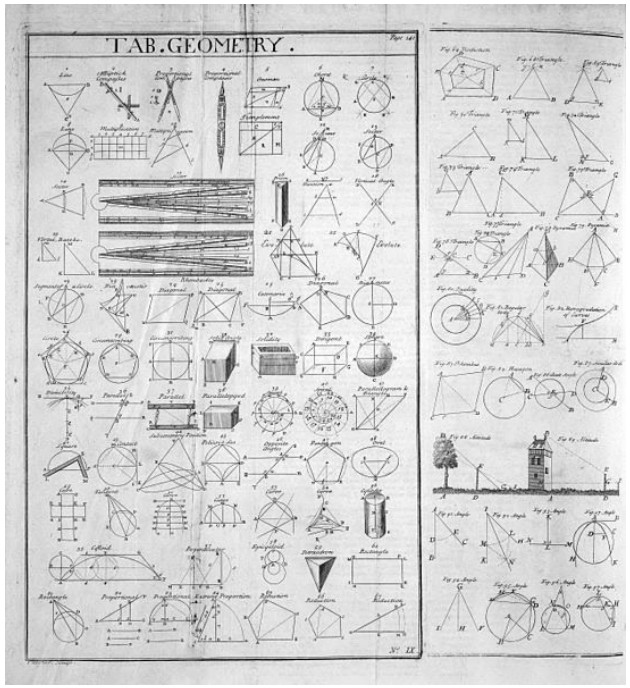
- Regarding robustness: imprecision of floating-point arithmetic
- FirstConvexHull possibly produces no valid polygon
- Graham and Jarvis always provide a valid polygon, but it may have minor defects

How to design Geometric Algorithm : Guidelines

1.) Eliminate degenerate cases ($\rightarrow$ *general position*)
- unique x (or y)-coordinates
- no three collinear points
- ...

2.) Adjust to degenerate inputs
- integrate into existing solutions
  (e.g., use lexicographic order if x-coordinates are not unique)
- may require special treament

3.) Implementation
- primitive operations (available in libraries?)
- robustness



Thank you!